

# Journal of Information Security and Applications

## BEdX25519: A Lightweight Authenticated Key Exchange Protocol for IoT Device-to-Device (D2D) Secure Communications based on Blockchain and Elliptic Curve Cryptography --Manuscript Draft--

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Manuscript Number:    | JISAS-D-25-02232                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Article Type:         | Full Length Article                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Section/Category:     | Cryptographic protection                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Keywords:             | IoT Security; Key Exchange Protocol; Mutual Authentication; Elliptic Curve Cryptography; Blockchain; Smart Contract                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Corresponding Author: | Rakibul Hasan<br>Bangladesh University of Business and Technology<br>BANGLADESH                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| First Author:         | Rakibul Hasan                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Order of Authors:     | Rakibul Hasan<br>Md Al Amin<br>Md Ibrahim Abdullah<br>A B M Shawkat Ali                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Abstract:             | <p>The Internet of Things (IoT) has transformed connectivity by enabling physical devices to communicate and exchange data across diverse domains such as healthcare, agriculture, education, and transportation. However, as IoT systems increasingly handle sensitive information, ensuring secure communication has become a critical challenge, particularly in device-to-device (D2D) scenarios. Traditional centralized security models suffer from single points of failure, making them prone to collapsing the entire system if the central authority becomes unavailable. In IoT device-to-device (D2D) communication, mutual authentication is a key challenge, as limited memory prevents devices from storing all trusted device credentials. To address these challenges, this paper proposes BEdX25519, a blockchain-based lightweight authenticated key exchange protocol tailored for secure IoT D2D communication without relying on centralized intermediaries. The protocol ensures that each device's unique identifier and public keys are securely stored on a decentralized, immutable blockchain ledger, making unauthorized tampering computationally infeasible. This design enables devices to retrieve each other's authenticated public keys directly from the blockchain, supporting secure session key generation via elliptic curve cryptography. The protocol was implemented on ESP32 devices to assess feasibility and performance. Experimental evaluations demonstrate its efficiency in terms of time and memory consumption. Security was validated through both informal analysis and formal verification using the Scyther tool. Additionally, real-world attack simulations confirmed the protocol's resilience against common threats such as replay, impersonation, and MITM attacks. Blockchain transaction costs were also evaluated across Ethereum (Layer 1) and Polygon (Layer 2) networks, affirming the protocol's cost-effectiveness. Overall, the results indicate that the proposed protocol is secure, lightweight, and practical for deployment in resource-constrained IoT environments.</p> |
| Opposed Reviewers:    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

**Anthony T.S. Ho, PhD**

Editor-in-Chief

Journal of Information Security and Applications

Elsevier

July 10, 2025

Dear Dr. Ho,

I wish to submit an original research paper entitled “**BEdX25519: A Lightweight Authenticated Key Exchange Protocol for IoT Device-to-Device (D2D) Secure Communications based on Blockchain and Elliptic Curve Cryptography**” to be considered for publication in the *Journal of Information Security and Applications*.

This paper addresses the problem of authentic key exchange between IoT devices to enable secure device-to-device communication. Traditional cryptographic protocols have proven to be unsuitable for such environments due to computational and memory constraints. Moreover, it remains uncertain whether blockchain can be effectively integrated with lightweight cryptographic techniques to support scalable and secure IoT deployments. In this study, we propose a combined approach using Blockchain technology and Elliptic Curve Cryptography (ECC), demonstrating that our protocol enables decentralized, authentic key exchange between IoT devices, eliminating the need for a central authenticator, while ensuring robustness against common security threats such as man-in-the-middle, replay, and modification attacks. Experimental results confirm that the proposed protocol provides an efficient, lightweight, and robust security solution, making it highly applicable for real-world IoT scenarios.

Since our study focuses on the design of a lightweight, authentic key exchange protocol for IoT systems, an area that aligns closely with the scope of *Journal of Information Security and Applications*, we believe it represents a strong fit for publication. We are confident that the findings will be highly relevant to your readership, as the proposed protocol effectively balances lightweight performance with strong cryptographic guarantees, addressing critical security needs for next-generation IoT applications.

The article “**BEdX25519: A Lightweight Authenticated Key Exchange Protocol for IoT Device-to-Device (D2D) Secure Communications based on Blockchain and Elliptic Curve Cryptography**” has not been published elsewhere, and it reflects original research conducted by its authors. None of the authors have any conflicts of interest to disclose concerning this study.

Thank you for your valuable time and consideration.

Sincerely,



**Rakibul Hasan, CEH (Practical), CC**

Research Assistant

Department of Computer Science and Engineering

Bangladesh University of Business and Technology

Email: rakibulhasanmrh@gmail.com

LinkedIn: <https://linkedin.com/in/rakibulhasanmrh>

# BEdX25519: A Lightweight Authenticated Key Exchange Protocol for IoT Device-to-Device (D2D) Secure Communications based on Blockchain and Elliptic Curve Cryptography

Rakibul Hasan<sup>a,\*</sup>, Md Al Amin<sup>b</sup>, Md Ibrahim Abdullah<sup>c</sup> and A B M Shawkat Ali<sup>a</sup>

<sup>a</sup>Department of Computer Science & Engineering, Bangladesh University of Business and Technology, Dhaka 1216, Bangladesh

<sup>b</sup>Department of Computer Science & Engineering, Colorado State University, Fort Collins, Colorado 80523, USA

<sup>c</sup>Department of Computer Science & Engineering, Islamic University, Kushtia 7003, Bangladesh

## ARTICLE INFO

### Keywords:

IoT Security  
Key Exchange Protocol  
Mutual Authentication  
Elliptic Curve Cryptography  
Blockchain  
Smart Contract

## ABSTRACT

The Internet of Things (IoT) has transformed connectivity by enabling physical devices to communicate and exchange data across diverse domains such as healthcare, agriculture, education, and transportation. However, as IoT systems increasingly handle sensitive information, ensuring secure communication has become a critical challenge, particularly in device-to-device (D2D) scenarios. Traditional centralized security models suffer from single points of failure, making them prone to collapsing the entire system if the central authority becomes unavailable. In IoT device-to-device (D2D) communication, mutual authentication is a key challenge, as limited memory prevents devices from storing all trusted device credentials. To address these challenges, this paper proposes BEdX25519, a blockchain-based lightweight authenticated key exchange protocol tailored for secure IoT D2D communication without relying on centralized intermediaries. The protocol ensures that each device's unique identifier and public keys are securely stored on a decentralized, immutable blockchain ledger, making unauthorized tampering computationally infeasible. This design enables devices to retrieve each other's authenticated public keys directly from the blockchain, supporting secure session key generation via elliptic curve cryptography. The protocol was implemented on ESP32 devices to assess feasibility and performance. Experimental evaluations demonstrate its efficiency in terms of time and memory consumption. Security was validated through both informal analysis and formal verification using the Scyther tool. Additionally, real-world attack simulations confirmed the protocol's resilience against common threats such as replay, impersonation, and MITM attacks. Blockchain transaction costs were also evaluated across Ethereum (Layer 1) and Polygon (Layer 2) networks, affirming the protocol's cost-effectiveness. Overall, the results indicate that the proposed protocol is secure, lightweight, and practical for deployment in resource-constrained IoT environments.

## 1. Introduction

The Internet of Things (IoT) is a vast network of interconnected devices equipped with sensors, software, and communication technologies that enable seamless data exchange over the Internet or other networks [1]. These devices play a crucial role in various domains, such as smart cities, healthcare, industrial automation, and connected vehicles, where they collect, process, and transmit sensitive information. However, the lack of robust security mechanisms makes IoT systems vulnerable to cyber threats, including unauthorized data access, device tampering, and malicious interceptions. If attackers exploit communication vulnerabilities, they can manipulate transmitted data, compromise device integrity, or gain control over critical infrastructure. Ensuring secure data communication is therefore essential to protect IoT networks from such threats while maintaining regulatory compliance and preventing financial and reputational damages.

\*Corresponding author



rakibulhasanmrh@gmail.com (R. Hasan); md.al\_amin@colostate.edu

(M.A. Amin); ibrahim@cse.iu.ac.bd (M.I. Abdullah);

shawkat.ali@bubt.edu.bd (A.B.M.S. Ali)

ORCID(s): 0000-0002-5106-9054 (R. Hasan); 0000-0003-1700-7201

(M.A. Amin); 0009-0009-2235-0910 (M.I. Abdullah); 0000-0001-9673-0080

(A.B.M.S. Ali)

Among different IoT communication paradigms, device-to-device (D2D) communication is emerging as a powerful mechanism that enables direct interactions between IoT devices without relying on centralized cloud servers or traditional network infrastructure [2, 3]. This decentralized approach enhances efficiency, reduces reliance on single points of failure, and facilitates low-latency communication, making it ideal for applications like smart transportation, industrial control systems (ICS), connected healthcare, and automation [4]. However, while D2D communication improves scalability and responsiveness, it also introduces significant security challenges, as devices must establish secure and authenticated channels in an environment where mutual trust is difficult to ensure. Attackers may attempt to intercept, manipulate, or impersonate devices, leading to potential security breaches and system disruptions.

One of the major security challenges in D2D communication is ensuring mutual authentication, as the absence of centralized authentication mechanisms makes it impractical for devices to maintain a list of trusted peers. Additionally, threats related to confidentiality, integrity, and authenticity pose serious concerns. Confidentiality risks arise when unauthorized entities gain access to sensitive data, integrity threats involve unauthorized modifications to data during

transmission, and authenticity threats occur when attackers impersonate legitimate devices or inject fraudulent data into the network. Addressing these challenges requires a robust and scalable security framework capable of securing communications in dynamic and decentralized IoT environments.

A fundamental aspect of secure communication is encryption, which ensures that transmitted data remains confidential and inaccessible to unauthorized entities. However, encryption requires a securely exchanged encryption key between communicating parties. The challenge lies in ensuring that this key exchange process is secure and protected from interception or manipulation by malicious adversaries. If an attacker compromises the encryption key during transmission, they can decrypt and manipulate sensitive data, leading to security breaches. Therefore, a robust key exchange mechanism is essential to establish secure communication channels, particularly in device-to-device (D2D) communication, where centralized key distribution is impractical.

Numerous key exchange mechanisms have been proposed to enable secure communication in IoT networks. Some of these rely on specialized hardware components such as Physical Unclonable Functions (PUFs) or Field Programmable Gate Arrays (FPGAs) to perform cryptographic operations, often in conjunction with centralized authentication servers[5, 6]. Other approaches eliminate centralized authorities but still depend on dedicated hardware, which may not be cost-effective or practical for all deployments[7, 8]. Additionally, with the advancement of quantum computing, several protocols have incorporated post-quantum cryptographic techniques to ensure future-proof security[9, 10]. However, such schemes are typically computationally intensive and are not well-suited for resource-constrained devices like the ESP32.

In our prior work[11], we proposed a blockchain-based key exchange mechanism for IoT device-to-device (D2D) communication. While the approach leveraged decentralized storage and public key access, it had some limitations. Specifically, the device ID was self-generated by the IoT device and submitted for registration, potentially leading to identity duplication. Furthermore, the key exchange process lacked a timestamp mechanism, making it susceptible to replay attacks. The update procedure for cryptographic keys was also not explicitly defined, raising concerns about long-term security and forward secrecy.

To address these shortcomings, this paper introduces a Lightweight Authenticated Key Exchange Protocol for IoT Device-to-Device (D2D) Secure Communication, combining blockchain technology with Elliptic Curve Cryptography (ECC). In the proposed protocol, each device's identity and public keys are stored on a blockchain through a smart contract, ensuring tamper-proof and decentralized access to critical authentication data. When two IoT devices wish to communicate, they retrieve each other's public keys from the blockchain and generate a shared secret key using the X25519 key exchange algorithm. This key is used for symmetric encryption to ensure confidentiality, integrity, and

authenticity during data transmission. To defend against replay attacks and message tampering, the protocol integrates timestamps and HMACs (Hash-based Message Authentication Codes) into each session. Devices verify these parameters before proceeding with communication. Furthermore, the shared secret key and public keys are not stored persistently on the device, mitigating risks in the event of key exposure. The protocol also supports secure key updates through digitally signed requests to the registration authority, enhancing forward secrecy without requiring additional hardware or central servers. This approach ensures a robust, decentralized, and resource-efficient security framework for IoT D2D communications.

The key contributions of this paper are summarized as follows:

- Developed a lightweight, mutually authenticated key exchange protocol specifically designed for secure IoT device-to-device communication, eliminating the need for centralized intermediaries by leveraging blockchain for decentralized authentication and public key verification.
- Performed informal security analysis to assess the protocol's resilience against cryptographic attacks and its support for forward secrecy, and formally verified its security properties using the Scyther tool.
- Evaluated the feasibility of the protocol by implementing it on ESP32 devices and analyzing performance in terms of time consumption, and memory utilization.
- Conducted real-world security testing by simulating common network attacks (e.g., MITM, replay, impersonation, and modification) in a controlled environment.
- Analyzed blockchain transaction costs associated with registration and key update operations to assess the financial and computational feasibility of deploying the protocol in practical IoT networks.

The remainder of this paper is structured as follows. Section 2 reviews relevant literature to highlight the necessity and context of the proposed work. Section 3 outlines the threat model associated with IoT device-to-device (D2D) communication. Section 4 describes the underlying cryptographic algorithms and mechanisms used in this study. Section 5 details the design of the proposed key exchange protocol. Section 6 provides a comprehensive security analysis of the protocol through both informal reasoning and formal verification techniques. Section 7 presents the implementation of the proposed protocol in an IoT environment and presents experimental results that validate its correctness. It also provides a comprehensive evaluation of the protocol's security robustness, feasibility, and performance, along with a cost analysis for maintaining the protocol on both Layer 1 and Layer 2 blockchain networks, highlighting its suitability for deployment in resource-constrained IoT environments. Section 8 discusses the findings, highlights the advantages

and limitations of the proposed approach, and compares it with related protocols. Finally, Section 9 concludes the paper and outlines directions for future research.

## 2. Related Works

Ensuring secure communication between Internet of Things (IoT) devices is a crucial challenge, particularly for resource-constrained environments. A fundamental requirement for secure communication is the establishment of a shared secret key between devices, enabling encryption for confidentiality and integrity. Several key exchange mechanisms have been proposed in the literature, including symmetric key cryptography, public key cryptography, physically unclonable functions (PUFs), and post-quantum cryptography. However, each of these approaches has its own limitations, motivating the need for a lightweight, scalable, and decentralized key exchange protocol.

Physically Unclonable Functions (PUFs) have been widely explored for IoT authentication and key exchange. Aman et al. [12] propose a PUF-based authentication mechanism that ensures security with minimal computational overhead. Similarly, Chatterjee et al. [13] integrate identity-based encryption (IBE), PUFs, and keyed hash functions to enhance key agreement. However, integrating PUFs into IoT devices is challenging and expensive due to hardware complexity and specialized chip requirements.

Li et al. [14] and Zheng et al. [5] present PUF-based mutual authentication schemes, leveraging certificate-less public key cryptography (CL-PKC) and reverse fuzzy extractors to enhance security. However, these methods introduce additional infrastructure overhead, as they require server-assisted computations. Roy et al. [6] and Gupta et al. [15] propose PUF-based authentication combined with lightweight cryptographic operations like XOR and hash functions. Despite their efficiency, these approaches still rely on intermediary verifiers, gateways, or internet connectivity, making them unsuitable for fully decentralized device-to-device (D2D) key exchange.

Furthermore, Fan et al. [8] propose a collaborative authentication and key exchange protocol using PUFs to enhance security in Industrial IoT (IIoT) environments. While PUFs offer a promising security mechanism, they are not yet widely adopted in commercial IoT devices due to high implementation costs and hardware constraints.

Public-key cryptography is often used for secure key exchange without requiring pre-shared secrets. Abdussami et al. [7] propose DEAC-IoT, a fog-enabled authentication scheme using Elliptic Curve Cryptography (ECC) to secure D2D communication. Although the protocol is validated using Scyther and Real-Or-Random (RoR) models, the reliance on Field Programmable Gate Arrays (FPGAs) increases costs and hardware complexity.

A significant challenge in deploying Public Key Infrastructure (PKI) in IoT is the scalability issue of managing certificates for large-scale IoT networks. The overhead of maintaining certificate issuance, renewal, and revocation poses implementation challenges in resource-constrained devices.

Furthermore, the cost of large-scale PKI deployment can be prohibitive, making certificate-based solutions unsuitable for low-power IoT applications.

Symmetric key cryptography is computationally efficient, making it attractive for IoT security. Ashraf et al. [16] propose a lightweight symmetric key exchange protocol implemented on Linux-based virtual systems. However, this approach requires a third-party server for managing key exchanges, introducing a single point of failure. Similarly, Rabiah et al. [17] and Ehui et al. [18] propose HMAC-based key exchange mechanisms that rely on gateway-assisted authentication, limiting their applicability for direct D2D communication.

Fan et al. [19] introduce SAKE\*, a lightweight Authenticated Key Exchange (AKE) protocol that minimizes computation and communication overhead. However, it still depends on a Trusted Authority (TA) for authentication, reducing its scalability and decentralization. The primary limitation of symmetric key cryptography is its dependence on a centralized authentication server, making it infeasible for fully decentralized D2D communication in IoT environments.

Post-quantum cryptography aims to counter threats posed by quantum computers. Selvakumar et al. [10] propose Crystals Kyber based scheme STUCK, a post-quantum key exchange mechanism designed for secure D2D communication. Similarly, Sarkar et al. [9] introduce a lattice-based key exchange protocol combined with blockchain technology. However, these approaches suffer from high computational overhead, making them impractical for low-power IoT devices.

Elliptic Curve Cryptography (ECC) provides a strong security-to-key-size ratio, making it well-suited for resource-constrained IoT devices [20]. Kodali and Nakoti [21] implement ECDH for IoT key exchange but do not address authentication, making the protocol vulnerable to Man-in-the-Middle (MITM) attacks [22]. Phimphinit et al. [23] enhance ECDH security by integrating a third-party trusted authority, but this introduces a single point of failure [24].

Several works propose identity-based authentication for ECDH key exchange. Arvandy et al. [25] integrate local gateways for authentication, while Salman et al. [26] and Shah et al. [27] employ identity-based cryptography for IoT authentication. However, these methods rely on centralized authentication servers, limiting their feasibility for fully decentralized IoT environments.

Gaba et al. [28] introduce Elliptic Curve Qu-Vanstone (ECQV)-based implicit certificates, but certificate-based authentication suffers from scalability issues, particularly in large-scale IoT deployments.

Blockchain technology offers a decentralized and tamper-proof alternative for IoT authentication and key exchange. Muth et al. [29] propose SmartDHX, which integrates Diffie-Hellman key exchange with blockchain-based authentication. However, Diffie-Hellman is computationally expensive, making it unsuitable for resource-constrained IoT devices.

El-Hajj et al. [30] utilize Ethereum blockchain and pre-shared keys for authenticated key exchange. While this method enhances security, the reliance on public-key encryption increases computational overhead. A lightweight pre-shared key strategy combined with blockchain authentication could be an effective approach to achieving secure, decentralized, and efficient key exchange in IoT networks.

Existing IoT key exchange mechanisms face several challenges, including hardware complexity in PUF-based solutions, reliance on central servers in symmetric cryptography, computational overhead in post-quantum cryptography, and scalability issues in certificate-based authentication. While ECDH provides a lightweight key exchange mechanism, its lack of authentication makes it vulnerable to active attacks.

Blockchain-based authentication offers a decentralized and tamper-proof approach to secure device-to-device key exchange. The combination of ECDH for lightweight key exchange and blockchain for decentralized authentication ensures secure, scalable, and efficient IoT communication without relying on trusted third parties or centralized intermediaries. The Elliptic Curve Diffie-Hellman (ECDH) protocol is widely used for lightweight key exchange, and its integration with blockchain further strengthens IoT security by preventing key compromise and unauthorized access. Among various elliptic curves, Curve25519 is a highly efficient and secure curve, specifically designed for fast computations and robust cryptographic security. Due to its elegant mathematical structure, Curve25519 is well-suited for resource-constrained IoT devices, providing an optimal balance between performance and security [31]. The key exchange protocol based on ECDH with Curve25519, known as X25519[32], is widely recognized for its efficiency and resistance to cryptographic attacks. This motivates our proposed approach, which integrates X25519 with blockchain authentication to achieve secure, mutual authentication in IoT D2D communication.

### 3. Threat Model

IoT device-to-device (D2D) communication enables direct interaction between connected devices without relying on centralized cloud services, helping to prevent a single point of failure. However, this communication model introduces several security threats that can compromise data integrity, confidentiality, and authentication. This section presents a structured threat model focusing on Man-in-the-Middle (MITM) attacks, which exploit vulnerabilities in communication protocols to intercept, manipulate, or forge messages. The key attack vectors analyzed include eavesdropping, impersonation, modification, and replay attacks. The attack topology illustrating these threats is shown in Figure 1.

**(i) Eavesdropping Attack:** Eavesdropping occurs when an attacker passively intercepts communication between two IoT devices to gain unauthorized access to exchanged data [33]. Since D2D communication often relies on wireless protocols, attackers can capture transmitted messages without altering them. This attack threatens the confidentiality

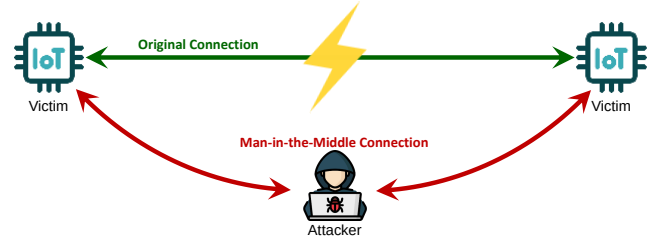


Figure 1: Attack topology

of sensitive information, such as cryptographic keys or authentication credentials.

**(ii) Modification Attack:** A modification attack occurs when an attacker intercepts and alters messages exchanged between IoT devices [34]. This enables the adversary to manipulate transmitted data, insert malicious commands, or modify cryptographic parameters to weaken security mechanisms. Since IoT networks often operate autonomously, compromised messages can lead to unintended actions or system failures.

**(iii) Replay Attack:** In a replay attack, an attacker captures and retransmits previously exchanged messages to deceive a device into accepting outdated or malicious requests [35, 36]. This attack can lead to unauthorized access, fraudulent key exchange, or disruption of normal device operations. Since IoT devices often communicate autonomously, they may not recognize duplicate messages without additional verification mechanisms.

**(iv) Impersonation Attack:** In an impersonation attack, an attacker attempts to pose as a legitimate device by intercepting communication and injecting fraudulent credentials [37]. This attack can be performed by capturing and replaying previously transmitted authentication messages or modifying device identifiers to trick a target into believing they are communicating with a trusted entity. This can lead to unauthorized access and data manipulation.

### 4. Preliminaries

This section discusses the fundamental components of the proposed protocol, which integrates blockchain technology and Elliptic Curve Cryptography (ECC) to enhance the security and integrity of device communications within an IoT network.

#### 4.1. X25519

X25519, introduced by Bernstein in 2006 [38], is an Elliptic Curve Diffie-Hellman (ECDH) key agreement protocol that utilizes Curve25519 as its elliptic curve basis. Curve25519 is recognized for its speed and security, leveraging the arithmetic of the Montgomery curve over the prime field  $\mathbb{F}_{2^{255}-19}$ . The curve is defined by the equation:

$$E : y^2 = x^3 + 486662x^2 + x$$

X25519 facilitates the secure exchange of cryptographic keys between parties, enabling the establishment of a shared secret over an insecure channel.

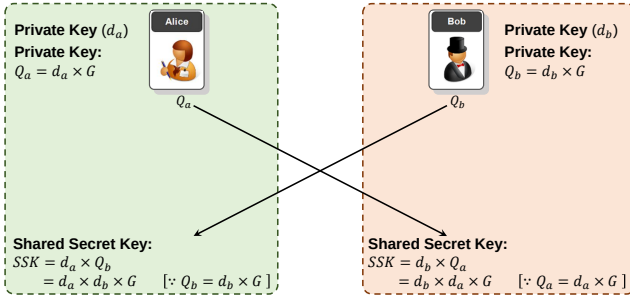


Figure 2: X25519 key exchange protocol.

In the X25519 protocol, each party generates a private key using a random number generator, selecting a 256-bit value. They then compute their respective public keys by performing elliptic curve scalar multiplication between their private key and the curve's base point. Upon exchanging public keys, both parties calculate a shared secret by again performing elliptic curve scalar multiplication, this time between their own private key and the received public key. This results in identical shared secret keys on both sides, which can subsequently be used for symmetric encryption to secure communications, as illustrated in Figure 2.

However, the X25519 protocol primarily protects against passive attacks, where an attacker can only observe the communication without interfering. It does not inherently defend against active attacks, where an attacker can intercept, modify, or inject messages into the communication channel. To mitigate this limitation, an additional authentication mechanism is required. Integrating authentication ensures that the communicating parties can verify each other's identities, thereby preventing man-in-the-middle and other active attacks.

#### 4.2. Ed25519

Ed25519 is a high-performance public-key signature system based on the Edwards-curve Digital Signature Algorithm (EdDSA). Developed by Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang, Ed25519 employs the elliptic curve Curve25519, which is known for its efficiency and security. The Edwards curve used in Ed25519 is defined by the equation  $-x^2 + y^2 = 1 + dx^2y^2$  over the prime field  $\mathbb{F}_{2^{255}-19}$ , where  $d = -121665/121666$ . Ed25519 offers several advantages, including fast signing and verification speeds, strong security properties, and resistance to common cryptographic vulnerabilities such as side-channel attacks. The key generation process in Ed25519 involves creating a 256-bit private key, from which a corresponding 256-bit public key is derived. The signing process uses the private key to produce a compact signature, which can then be verified using the public key. Due to its robustness and efficiency, Ed25519 is widely adopted in various applications, including secure communications, digital certificates, and cryptographic protocols, making it a popular choice for modern cryptographic needs.

#### 4.3. HMAC

HMAC (Hash-based Message Authentication Code) is a widely used cryptographic technique that ensures both data integrity and authentication by combining a cryptographic hash function (e.g., SHA-256, SHA-1) with a secret key. It operates by processing the secret key through padding and hashing in two stages: inner hashing and outer hashing. Initially, the key is XORed with an inner padding constant (ipad) and concatenated with the message, followed by hashing. The resulting hash is then combined with an outer padding constant (opad) and rehashed to produce the final HMAC value. This structure enhances security by preventing length extension attacks, making HMAC a preferred choice for secure communication, API authentication, digital signatures, and cryptographic protocols like TLS and IPsec. The strength of HMAC lies in its resistance to key recovery attacks and its ability to verify message authenticity between communicating parties without exposing the original key.

#### 4.4. Blockchain

Blockchain is a decentralized and distributed ledger technology that ensures secure and immutable record-keeping of transactions across a network of computers. Each block in the blockchain contains a list of transactions and a cryptographic hash of the previous block, linking them together in a chain. This structure ensures that once data is recorded, it cannot be altered or deleted, providing data integrity and transparency. Blockchain operates on a consensus mechanism, where network participants (nodes) validate and agree on the authenticity of transactions before they are added to the ledger. This eliminates the need for a central authority and reduces the risk of fraud and tampering.

#### 4.5. Smart Contract

A smart contract is a self-executing contract with the terms of the agreement directly written into lines of code. These contracts run on blockchain networks, ensuring that once deployed, they operate in a decentralized and tamper-proof manner. The code and the agreements contained therein are distributed across the network, which means they are not controlled by any single entity, adding a layer of trust and transparency.

Smart contracts automatically enforce and execute the terms of an agreement when predefined conditions are met. This automation reduces the need for intermediaries, thereby decreasing transaction costs and speeding up processes.

The functionality of smart contracts extends beyond simple transactions. They can be used to create decentralized applications (dApps) and automate complex multi-party agreements. Their applications are vast, including areas like supply chain management, where they can track the provenance of goods, in healthcare for secure and transparent handling of medical records, and in the financial sector for automating trading and settlement processes.

Overall, smart contracts enhance the capabilities of blockchain technology by providing a secure, transparent,

**Table 1**  
Definition of Notations.

| Notation | Definition            |
|----------|-----------------------|
| $ID$     | Device identification |
| $P$      | X25519 public key     |
| $Q$      | X25519 private key    |
| $M$      | Ed25519 public key    |
| $N$      | Ed25519 private key   |
| $K$      | Shared secret key     |
| $T$      | Timestamp             |

**Note:** The subscript will indicate the device name.

and efficient method to manage and automate agreements and processes across various industries.

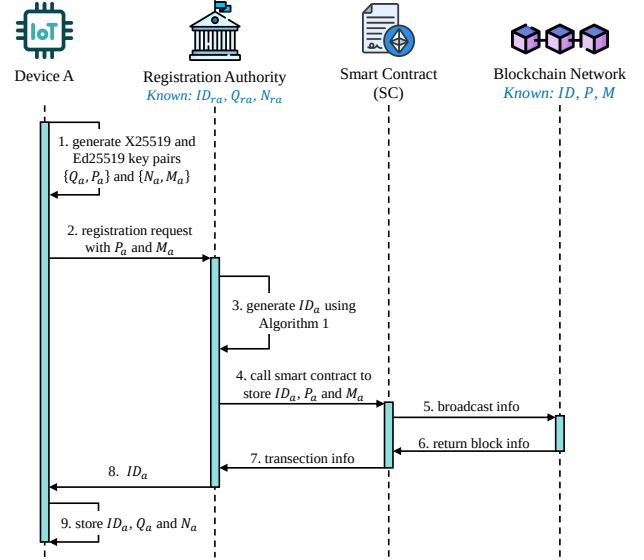
## 5. Proposed Protocol

This section introduces the proposed protocol, which integrates Blockchain technology and Elliptic Curve Cryptography (ECC) to enhance security in device communication. The protocol employs the existing key exchange protocol X25519 for key exchange and uses blockchain to securely store pre-shared public keys. The Ed25519 digital signature algorithm is utilized to ensure the integrity and authenticity of sensitive information during updates. Prior to joining the network, each device must undergo a registration process as detailed in subsection 5.1. For secure communication, any two registered devices generate a shared secret key, which is then used for message encryption via symmetric key cryptography, as outlined in subsection 5.2. Periodic key updates are crucial for maintaining security; therefore, devices periodically generate new X25519 key pairs. Additionally, if a key is suspected to be compromised, the device promptly updates its X25519 key pair. Ed25519 digital signature algorithm ensures the authenticity of these updates, as described in subsection 5.3. This protocol leverages the strengths of blockchain and ECC to provide a robust framework for secure and efficient device communication. Table 1 provides the definition of notations for this section.

### 5.1. Device Registration

During the device registration process, the IoT device generates two cryptographic key pairs for X25519 and Ed25519. The private keys of both X25519 and Ed25519 are securely stored in the device's non-volatile memory, such as EEPROM, ensuring persistence even when the device is powered off. The device then transmits the corresponding public keys to the registration authority. Upon receiving these public keys, the registration authority generates a unique identifier (ID) for the device using a predefined algorithm (Algorithm 1). This unique ID, along with the device's public keys, is stored on the blockchain through a smart contract. The utilization of blockchain technology ensures that the registration data is immutable and transparent, enhancing the overall security and integrity of the registration process. Once the information is successfully

recorded on the blockchain, the registration authority sends the unique ID back to the device, which then stores this ID in its non-volatile memory. This comprehensive approach ensures secure device authentication and traceability within the IoT ecosystem.



**Figure 3:** Device registration process.

Figure 3 visually represents the steps involved in the device registration process, outlined as follows:

**Step 1:** The IoT device generates the X25519 and Ed25519 key pairs  $\{Q_a, P_a\}$  and  $\{N_a, M_a\}$ , respectively.

**Step 2:** The device requests registration by sending the public keys  $P_a$  and  $M_a$  to the Registration Authority (RA).

**Step 3:** The RA generates a unique ID for the device using Algorithm 1.

**Step 4-7:** The RA invokes a smart contract to store the device's ID and public keys on the blockchain. Upon successful transaction, confirmation is returned.

**Step 8:** The RA assigns the unique ID to the device.

**Step 9:** The device stores the received ID and its generated public keys in non-volatile memory.

This approach ensures that all devices are uniquely identified and that their cryptographic materials are securely anchored in a decentralized and tamper-proof ledger, providing a strong foundation for secure communication and lifecycle management across the IoT network.

### 5.2. Shared Secret Key Generation

Each IoT device initially broadcasts its unique identifier (ID) to enable mutual recognition. Once the target device's ID is known, the initiator begins the key exchange process by retrieving the corresponding public key from the blockchain via a smart contract. Using the retrieved public key and its own private key, the initiator performs elliptic curve scalar multiplication as defined in the X25519 algorithm to derive a shared secret key. To authenticate the key exchange and ensure data integrity, the initiator generates a Hash-based Message Authentication Code (HMAC) over the current

**Algorithm 1** Unique ID generation**Require:** X25519 public key ( $P$ ), Ed25519 public key ( $M$ )**Ensure:** Unique device id ( $ID$ )

```

 $T \leftarrow \text{timestamp}()$ 
 $D \leftarrow (P \parallel M \parallel T)$   $\triangleright$  Concatenate  $P$ ,  $M$ , and  $T$ 
 $H \leftarrow \text{sha256}(D)$ 
 $h_1 \leftarrow H[0 : 7]$   $\triangleright$  First eight byte of  $H$ 
 $h_2 \leftarrow H[8 : 15]$   $\triangleright$  Second eight byte of  $H$ 
 $h_3 \leftarrow H[16 : 23]$   $\triangleright$  Third eight byte of  $H$ 
 $h_4 \leftarrow H[24 : 31]$   $\triangleright$  Fourth eight byte of  $H$ 
 $ID \leftarrow (h_1 \oplus h_2 \oplus h_3 \oplus h_4)$ 

```

timestamp and its own ID using the shared secret key. The initiator then transmits the timestamp, its ID, and the generated HMAC to the target device.

Upon receiving this data, the target device first validates the received timestamp by checking the difference between the received timestamp and the current time. If the difference is within the maximum transmission delay, the target device proceeds to retrieve the public key of the initiator from the blockchain via the smart contract and computes the shared secret key using its private key and the retrieved public key. To verify the integrity and authenticity of the received data, the target device regenerates the HMAC from the received timestamp and ID using the derived shared secret key. If the computed HMAC matches the received HMAC, the target device generates another HMAC using the received timestamp and the current timestamp. It then responds to the initiator by sending the received timestamp, the newly generated timestamp, and the computed HMAC.

Upon receiving the response, the initiator validates the received timestamp to ensure data freshness and verifies the integrity of the data by computing the HMAC using the shared secret key and checking for a match with the received HMAC. If both values match, the session is successfully established, with the shared secret key computed in Step 5 (initiator) and Step 11 (responder) being identical on both devices. The derived shared secret key can then be used as a symmetric encryption key, enabling secure and authenticated communication between the devices within the IoT network. This process ensures mutual authentication, protects against replay attacks, and establishes a secure session using blockchain-based public key retrieval.

Figure 4 visually represents the steps involved in the secret key sharing process, outlined as follows:

**Step 1 – 4:** Device A identifies Device B using its broadcasted identifier ( $ID_b$ ) and calls a smart contract to retrieve Device B's X25519 public key ( $P_b$ ) from the blockchain. The smart contract returns  $P_b$  to Device A.

**Step 5:** Device A computes the shared secret key  $K_a$  using its own private key and the public key  $P_b$ . Using  $K_a$ , Device A generates an HMAC  $H_a$  over the current timestamp  $T_a$  and its own ID ( $ID_a$ ).

**Step 6:** Device A sends the tuple  $\{T_a, ID_a, H_a\}$  to Device B to initiate the key exchange.

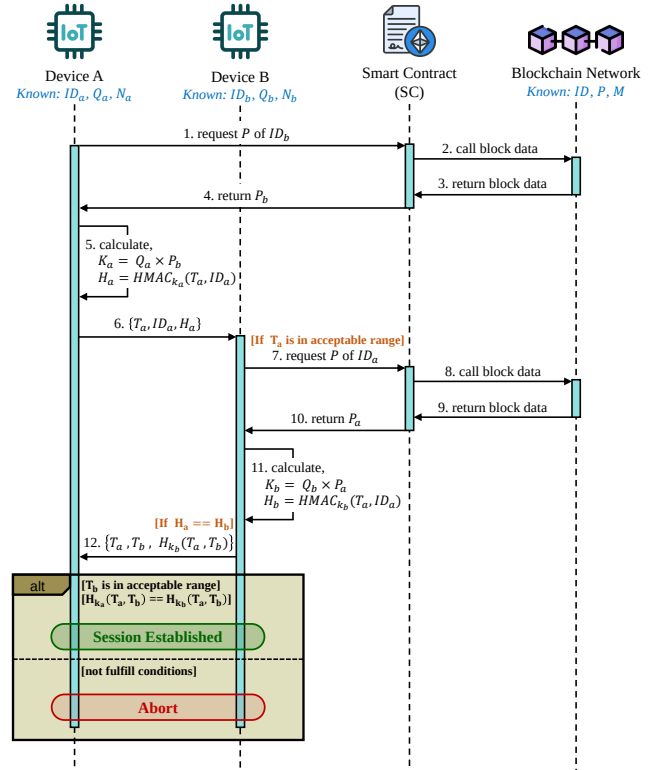


Figure 4: Key exchange process.

**Step 7–10:** Upon receiving the message, Device B verifies whether  $T_a$  falls within the acceptable delay window based on its local clock. If valid, Device B calls the smart contract to retrieve the public key of Device A ( $P_a$ ) using the received  $ID_a$ . The smart contract returns  $P_a$  to Device B.

**Step 11:** Device B computes the shared secret key  $K_b$  using its private key and  $P_a$ . Using  $K_b$ , Device B generates an HMAC value  $H_b$  from  $T_a$  and  $ID_a$ .

**Step 12:** Device B compares the generated HMAC  $H_b$  with the received HMAC value  $H_a$ . If they match, Device B generates a new HMAC over the received timestamp  $T_a$  and its current timestamp  $T_b$  using  $K_b$ . It then responds by sending  $\{T_a, T_b, H_{K_b}(T_a, T_b)\}$  back to Device A. Upon receiving the response, Device A checks the freshness of  $T_b$  and verifies that the received HMAC  $H_{K_b}(T_a, T_b)$  matches its locally generated HMAC using  $K_a$ . If both checks pass, the session is successfully established. Otherwise, the key exchange process is aborted.

### 5.3. Update X25519 Key Pair

To update its X25519 key pair, the device first generates a new X25519 key pair and signs the new X25519 public key and ID using its Ed25519 private key. It then sends its unique ID, the signature, timestamp, and the signed X25519 public key to the registration authority. The authority validates the received timestamp, and verifies the signature to confirm the request's authenticity. If the difference between the received timestamp and the current time is within the

maximum transmission delay, and the signature is valid, the authority updates the device's public key information on the blockchain through the smart contract. After the update is successfully recorded, the authority signs a confirmation message and sends it, along with the signature and timestamp, back to the device. Upon receiving this confirmation, the device validates the received timestamp to ensure data freshness. If the confirmation is valid, then retrieves the authority's signature verification key, verifies the signature, and if the verification is successful, updates its old X25519 private key with the new private key in its local storage. This process ensures a secure and authenticated update of cryptographic keys, enhancing the security and integrity of communications within the IoT network.

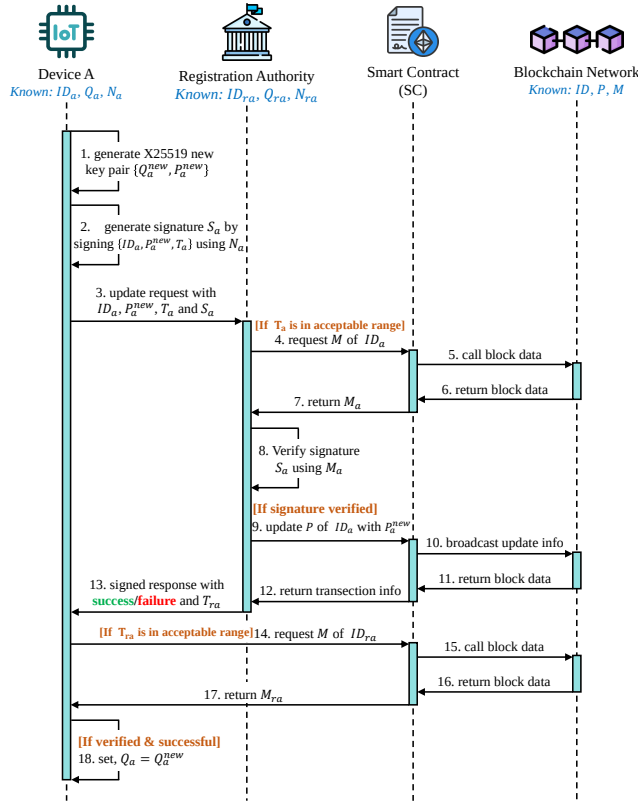


Figure 5: X25519 key update.

Figure 5 visually represents the steps involved in the X25519 key pair update process, outlined as follows:

**Step 1:** Device A generates a new X25519 key pair:  $\{Q_a^{new}, P_a^{new}\}$ .

**Step 2:** Device A signs its ID  $ID_a$ , new public key  $P_a^{new}$ , and timestamp  $T_a$  using its Ed25519 private key  $N_a$  to produce signature  $S_a$ .

**Step 3:** Device A sends a key update request to the Registration Authority (RA), including  $\{ID_a, P_a^{new}, T_a, S_a\}$ .

**Step 4–7:** The RA checks whether the timestamp  $T_a$  is within an acceptable delay. If acceptable, the RA invokes a smart contract to retrieve Device A's Ed25519 public key  $M_a$  from the blockchain using  $ID_a$ .

**Step 8:** The RA verifies the signature  $S_a$  using  $M_a$ .

**Step 9–12:** If verification succeeds, the RA calls a smart contract to update the blockchain with  $P_a^{new}$  and invalidate the old key. The transaction confirmation is received.

**Step 13:** The RA signs the update result ("success" or "failure") along with a timestamp  $T_{ra}$  and sends it to Device A. Device A checks if  $T_{ra}$  is within the valid time window.

**Step 14–17:** Device A retrieves the RA's public key  $M_{ra}$  from the blockchain via smart contract using  $ID_{ra}$ .

**Step 18:** Device A verifies the RA's signature. If valid and the result is "success," it updates its locally stored X25519 private key to  $Q_a^{new}$ .

## 6. Security Analysis

To evaluate the robustness and resilience of the proposed protocol, we conducted a comprehensive security analysis against common security threats. For this purpose, we assume that the blockchain, smart contract, and the APIs used to interact with the smart contract from IoT devices are secure and free from vulnerabilities. The analysis is performed in two phases: informal analysis and formal verification. The informal analysis assesses how well the protocol defends against common attack vectors such as man-in-the-middle (MITM), replay, impersonation, and modification attacks by examining the cryptographic constructs and system design. In addition, we perform a formal security analysis using an automated protocol verification tool to rigorously verify the correctness of the protocol's security properties. This dual-phase evaluation ensures both theoretical soundness and practical resilience of the proposed key exchange mechanism within the IoT environment.

### 6.1. Informal Analysis

We analyze the security of the proposed protocol by evaluating its resilience against common security threats. The proposed protocol ensures a high level of security by leveraging cryptographic mechanisms and blockchain technology to prevent various cyber threats, including eavesdropping, impersonation, modification, replay, and man-in-the-middle (MITM) attacks. The combination of X25519 key exchange, blockchain-based public key retrieval, Ed25519 digital signatures, and integrity verification mechanisms contributes to a robust and tamper-resistant communication framework for IoT devices.

To establish a shared secret key securely, the protocol employs the X25519 key exchange algorithm, which is widely recognized for its efficiency and strong security properties. However, a direct exchange of public keys between communicating devices is susceptible to MITM attacks, where an attacker could intercept and modify the exchanged keys. To mitigate this risk, the public keys of all devices are securely stored in the blockchain. Each device retrieves the corresponding public key of its communication partner directly from the blockchain, ensuring authenticity and integrity. This eliminates the need for direct public key exchange between devices, thereby preventing potential MITM attacks.

Replay attacks are effectively mitigated by incorporating timestamps and Hash-based Message Authentication Code (HMAC) values in each key exchange process. The timestamp serves as a freshness indicator, ensuring that the transmitted data is recent and not reused by an adversary. This mechanism helps to detect and discard any replayed messages. The HMAC, on the other hand, provides data integrity verification by ensuring that the transmitted data has not been altered during transit. Any unauthorized modification of the data would result in an HMAC mismatch, leading to the rejection of the communication attempt.

Impersonation attacks are effectively prevented by the protocol's use of blockchain-stored public keys. Instead of exchanging public keys directly, devices transmit their unique identifiers (IDs) during the key exchange process. Using these identifiers, they retrieve the corresponding public keys from the blockchain. This approach ensures that an attacker cannot impersonate a legitimate device since they would lack the private key corresponding to the stored public key. Even if an adversary attempts to use a valid device ID, they cannot generate the correct cryptographic credentials required for authentication, thereby preventing impersonation attacks.

To achieve perfect forward secrecy, the protocol incorporates a periodic key update mechanism. Each device generates a new X25519 key pair at regular intervals and updates its corresponding public key in the blockchain. This ensures that even if an attacker compromises a previous session key, they cannot decrypt past communications since new session keys are generated dynamically. The continuous renewal of cryptographic keys enhances long-term security and mitigates the risk of key compromise over time.

To prevent unauthorized key update requests, the protocol employs the Ed25519 digital signature algorithm, which is both secure and efficient. Each key update request includes a timestamp and a digital signature, ensuring data freshness and authenticity. The timestamp helps prevent replay attacks by ensuring that outdated update requests are not accepted. The digital signature, signed using the private key of the requesting device, guarantees data integrity and authenticity. Since the registration authority verifies the signature before processing the update request, any unauthorized attempts to modify or replace public keys are effectively thwarted.

By integrating these security mechanisms, the proposed protocol effectively protects against common cyber threats, ensuring secure and tamper-resistant IoT communication. The blockchain-based public key management eliminates trust dependencies on centralized authorities, while cryptographic techniques safeguard data integrity, confidentiality, and authenticity. Through these measures, the protocol establishes a secure, decentralized, and scalable authentication framework for IoT device-to-device communication.

## 6.2. Formal Verification

To formally analyze the security properties of the proposed key exchange protocol, we employ Scyther[39], a well-recognized tool for automated cryptographic protocol

verification. Scyther systematically analyzes possible attack vectors under an unbounded number of sessions and adversary capabilities defined by Dolev-Yao assumptions.

### 6.2.1. Protocol Specification in Scyther

Under the assumption that the blockchain, smart contract, and the APIs used to interact with the smart contract from IoT devices are secure and free from vulnerabilities, our verification focuses solely on the communication between two devices. The proposed key exchange protocol involves two primary entities: IoT Device *A* and *B*, which establish a shared session key through cryptographic exchanges. The protocol ensures key confidentiality, integrity, and mutual authentication.

The formal representation of the protocol was implemented using Scyther's Security Protocol Description Language (SPDL) to analyze its security properties, as illustrated in Figure 6. The protocol guarantees that only *A* and *B* can derive the shared secret key, preventing unauthorized access by adversaries.

We claim the verification of the following security properties:

- **Secret:** Ensures that the session key remains confidential and is only shared between the intended parties.
- **Liveness:** Confirms that both *A* and *B* actively participated in the protocol execution, preventing adversary-driven false interactions.
- **Weak Agreement:** Validates that *A* and *B* mutually authenticate each other and agree on the exchanged session key.
- **Non-Injective Agreement:** Ensures that at least one valid protocol run exists where *A* and *B* successfully establish the session key.
- **Non-Injective Synchronization:** Confirms that the message flow between *A* and *B* follows the expected sequence without manipulation.

### 6.2.2. Verification Results

After analyzing the proposed key exchange protocol using the Scyther tool, The verification results, confirm that all five claimed properties were successfully verified. The successful verification of all claimed security properties, as illustrated in Figure 7, demonstrates that our proposed key exchange protocol is resilient against major cryptographic attacks, including replay attacks, man-in-the-middle (MITM) attacks, impersonation, and key compromise attacks. This confirms the protocol's robustness in ensuring secure communication in IoT environments.

## 7. Experimental Evaluation

This section presents a comprehensive evaluation of the proposed protocol through experimental testing in a controlled IoT environment. The objective of this evaluation is to

```

usertype TimeStamp;
hashfunction HMAC;
function SSK;

protocol KeyExchange(A, B)
{
  role A
  {
    fresh Ta: TimeStamp;
    var Tb: TimeStamp;
    fresh Qa, Pb: Data;
    send_1(A, B, Ta, HMAC({Ta}k(A,B)));
    rcv_2(B, A, Ta, Tb, HMAC({Ta,Tb}k(B,A)));
    claim(A, Secret, SSK(Qa, Pb));
    claim(A, Alive);
    claim(A, Weakagree);
    claim(A, Niagree);
    claim(A, Nisynch);
  }

  role B
  {
    fresh Tb: TimeStamp;
    var Ta: TimeStamp;
    fresh Qb, Pa: Data;
    rcv_1(A, B, Ta, HMAC({Ta}k(A,B)));
    send_2(B, A, Ta, Tb, HMAC({Ta,Tb}k(B,A)));
    claim(B, Secret, SSK(Qb, Pa));
    claim(B, Alive);
    claim(B, Weakagree);
    claim(B, Niagree);
    claim(B, Nisynch);
  }
}

```

Figure 6: Protocol description in Scyther

validate the protocol's functionality, security, and efficiency under real-world conditions. The evaluation is structured into two main parts: the Experimental Setup and the Results.

### 7.1. Experimental Setup

This subsection describes the experimental setup employed to evaluate the performance and effectiveness of the proposed protocol. The testing environment consisted of multiple IoT devices, a Smart Contract implemented on a local blockchain infrastructure, and a Registration Authority device. The IoT devices were configured to simulate real-world communication scenarios, each equipped with cryptographic libraries to implement the X25519 key exchange and Ed25519 signing algorithms. The Smart Contract was responsible for securely storing and managing public keys on the blockchain, while the Registration Authority handled the registration and key update requests from the IoT devices. The devices were connected over a test network, as shown in Figure 8, allowing us to test key registration, exchange, and update processes.

Scyther results : verify

| Claim       |   |                |                  | Status | Comme                |
|-------------|---|----------------|------------------|--------|----------------------|
| KeyExchange | A | KeyExchange,A1 | Secret Ka(Qa,Pb) | OK     | Verified No attacks. |
|             |   | KeyExchange,A2 | Alive            | OK     | Verified No attacks. |
|             |   | KeyExchange,A3 | Weakagree        | OK     | Verified No attacks. |
|             |   | KeyExchange,A4 | Niagree          | OK     | Verified No attacks. |
|             |   | KeyExchange,A5 | Nisynch          | OK     | Verified No attacks. |
|             | B | KeyExchange,B1 | Secret Kb(Qb,Pa) | OK     | Verified No attacks. |
|             |   | KeyExchange,B2 | Alive            | OK     | Verified No attacks. |
|             |   | KeyExchange,B3 | Weakagree        | OK     | Verified No attacks. |
|             |   | KeyExchange,B4 | Niagree          | OK     | Verified No attacks. |
|             |   | KeyExchange,B5 | Nisynch          | OK     | Verified No attacks. |

Done.

Figure 7: Protocol verification result

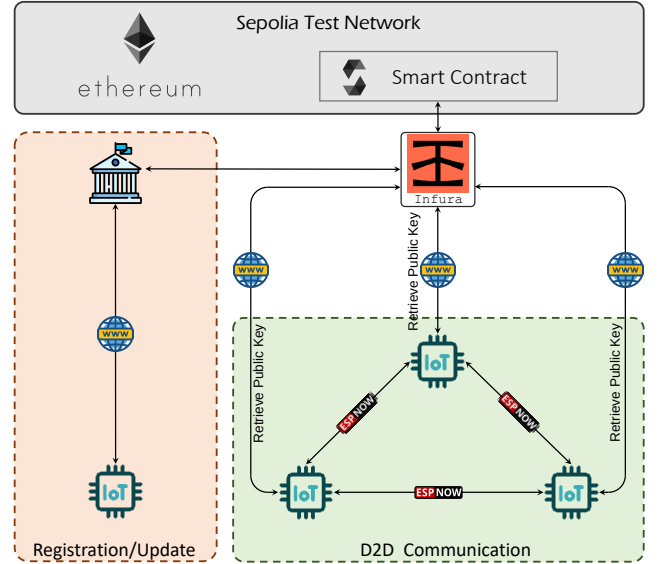


Figure 8: Protocol test network diagram.

#### 7.1.1. Smart Contract

To ensure secure and tamper-proof management of public keys, we developed a smart contract using Solidity, which facilitates interaction with the Ethereum blockchain. The smart contract includes four primary functions: *register()*, *update()*, *getPubKey()*, and *getSigKey()*, allowing devices to securely register, update, and retrieve public keys. Additionally, a modifier *onlyRA()* is implemented to restrict access to the *register()* and *update()* functions, ensuring that only the Registration Authority (RA) has the privilege to register new devices and update their information, thereby preventing unauthorized modifications.

The *register()* function allows the RA to store a device's public key on the blockchain during the initial registration process, while the *update()* function enables the RA to modify a registered device's public key when a key update request is received. The *getPubKey()* function is designed to

**Table 2**  
Registration Authority Device Specification

| Component | Specification                |
|-----------|------------------------------|
| CPU       | Intel Core i5 7th Genetation |
| Frequency | up to 3.1 GHz                |
| RAM       | 8 GB                         |
| OS        | Windows 10 Pro               |

retrieve the stored public key of a device, ensuring that key exchange processes can be securely performed. Similarly, the *getSigKey()* function allows devices to obtain the stored signature verification key for authentication purposes.

The smart contract was deployed on the Sepolia Testnet using the Remix Online IDE, a widely used Ethereum development environment. The deployment and subsequent interactions with the contract were facilitated through MetaMask, which was used to sign blockchain transactions securely. By leveraging blockchain technology, this smart contract ensures the integrity, authenticity, and immutability of registered and updated public keys, providing a secure foundation for cryptographic key management in the proposed protocol.

### 7.1.2. Registration Authority

To manage device registration and key updates securely, we implemented a Registration Authority (RA) server using the Flask framework, deployed on a local computer with the configuration shown in Table 2. The RA server is responsible for handling registration requests from IoT devices, processing the registration by storing the device ID and public keys on the blockchain via a smart contract. Additionally, it processes key update requests by verifying the authenticity of the requesting device before updating the corresponding public key on the blockchain.

To interact with the Ethereum blockchain, we utilized the Python *Web3* library along with *Infura API*, enabling seamless communication with the blockchain network. Through this integration, the RA can securely register new devices, retrieve stored public keys, and update existing keys while ensuring the integrity and authenticity of the data. Furthermore, we employed the *Cryptography* library to perform cryptographic operations, including key pair generation, signing, and verification, which are essential for securing the registration and key update processes.

By implementing the RA server, we establish a trust anchor within our protocol, ensuring that only authorized devices can register and update their public keys on the blockchain. This approach enhances security by preventing unauthorized modifications while enabling a decentralized and verifiable key management system for secure IoT communication.

### 7.1.3. IoT Device

For the implementation of the proposed protocol, we utilized three ESP32 devices, each functioning as an independent IoT device. The specifications of these devices are

**Table 3**  
IoT Device Specification

| Component       | Specification                |
|-----------------|------------------------------|
| Microcontroller | Xtensa® 32-bit LX6 Dual-core |
| Clock Speed     | Up to 240 MHz                |
| SRAM            | 520 KB                       |
| Flash Memory    | 4MB (external SPI flash)     |
| Wi-Fi           | 802.11 b/g/n (2.4 GHz)       |
| Bluetooth       | Bluetooth 4.2 BR/EDR and BLE |

detailed in Table 3. To facilitate device-to-device communication, we employed the ESP-NOW protocol, a low-latency wireless communication protocol that allows direct data exchange between ESP32 devices without requiring an internet connection. However, to interact with the blockchain, the devices were also connected to a Wi-Fi router to enable internet access.

To retrieve the public keys of other devices, each IoT device interacts with the Ethereum smart contract by calling smart contract functions through the Infura API. This ensures secure and efficient key retrieval from the blockchain without requiring direct access to an Ethereum node. Additionally, cryptographic operations, such as key generation, signing, and verification, were implemented using the Crypto library, ensuring secure key exchange and authentication.

The firmware for the IoT devices was developed in C++ and uploaded using Arduino IDE, enabling the devices to handle blockchain interactions, secure communication, and cryptographic computations seamlessly. By integrating ESP32 devices with both local wireless communication (ESP-NOW) and internet-based blockchain interaction, the implementation ensures secure and authenticated key management, enhancing the overall security and reliability of the proposed system.

## 7.2. Results

After implementing our proposed protocol in the testing environment, we conducted a thorough analysis through four key assessments: Validation Testing, Performance Testing, Security Testing, and Cost Analysis. The validation testing aimed to confirm the functional correctness of the protocol, focusing on the successful establishment of secure key exchanges and verifying the accuracy and integrity of the key update mechanism. This ensured that the protocol behaved as expected under typical operating conditions.

The performance testing evaluated the protocol's computational efficiency, measuring parameters such as execution time and memory usage during the key registration, exchange, and update processes. These metrics were collected from real ESP32 devices to assess the feasibility of the protocol on resource-constrained IoT hardware.

In addition to functionality and efficiency, we conducted security testing by simulating real-world attacks in a controlled environment. These simulations included common

**Table 4**  
Shared secret key generation.

| Device   | Key   | Value                                                             |
|----------|-------|-------------------------------------------------------------------|
| A        | $Q_a$ | d8313fb5a1fe0c4ca798a76aacc9922e6be06772ba44033e9920088658b5be6b  |
|          | $P_a$ | 88cf7d7f7f42aab12940da598f203264e52ef640db0eb8cc85050ac125ca5c39  |
| B        | $Q_b$ | a8bad0b7b0d07114eb2bccbd023cce4e507e533a568acb09e244e90885499461  |
|          | $P_b$ | a73b96bdbbe95feb81bd1de6bd08986645e5f1d2860970b4c1af320e9d015c020 |
| C        | $Q_c$ | 287d9d6fa0b75051220188bf5f5e79fe1c95e3de7c3c1c64689258b0f92a9c73  |
|          | $P_c$ | 115399051d09a2a878298ede36f26855aaece53dcb9d2be49d3a395297f8ab04  |
| A-B pair | $K_a$ | 26166e5baf27f4381f9a76b9b7ff190d3b8d68fe67a6d289a236398aefe40c6b  |
|          | $K_b$ | 26166e5baf27f4381f9a76b9b7ff190d3b8d68fe67a6d289a236398aefe40c6b  |
| A-C pair | $K_a$ | 9d03e3a36e94219c7681a130f0dad61c62223917b3bcc2ed2668efaaf8c5c70b  |
|          | $K_c$ | 9d03e3a36e94219c7681a130f0dad61c62223917b3bcc2ed2668efaaf8c5c70b  |

threats such as man-in-the-middle (MITM), replay, impersonation, and modification attacks. The protocol's response to each scenario was analyzed to evaluate its resilience and trustworthiness.

Finally, cost analysis was performed to assess the financial and computational feasibility of deploying the protocol in practice. This included measuring blockchain transaction fees on both testnet and mainnet environments (Ethereum and Polygon).

### 7.2.1. Validity Testing

In the validation testing phase, we assessed the functionality and effectiveness of the proposed protocol by verifying its ability to facilitate secure key exchanges and ensure successful key updates. The initial step involved registering three IoT devices, referred to as Device A, Device B, and Device C. Following the successful completion of the registration process, we initiated the key exchange procedure between these devices. This process aimed to establish a shared secret key, critical for secure communication. As shown in Table 4, each pair of devices successfully generated and exchanged shared secret keys.

For this validation test,  $Q_a$ ,  $Q_b$  and  $Q_c$  represent the private keys of Device A, Device B and Device C, respectively, which are securely stored locally on each device. Correspondingly,  $P_a$ ,  $P_b$  and  $P_c$  are the pre-shared public keys for Device A, Device B and Device C, stored on the blockchain during the registration process. Upon retrieving each other's public keys, both devices used these to compute the shared secret keys. During communication between the A – B device pair, Device A and Device B generate shared secret keys  $K_a$  and  $K_b$  respectively, which are identical. This confirms that both devices successfully derived the same shared secret key, validating the protocol's ability to securely exchange keys between the two devices.

Furthermore, during communication between the A – C device pair, Device A and Device C generate shared secret keys  $K_a$  and  $K_c$  respectively, which are also identical, but different from the key generated by the A – B device pair. This ensures that each device pair generates a unique shared secret key, maintaining the confidentiality of communication

within each pair and confirming the protocol's capability to generate distinct shared keys for different device pairs.

To further validate the protocol, we tested the key update mechanism. We triggered Device A to update its X25519 key pair. Device A successfully generated a new key pair and updated the corresponding information on the blockchain through the Registration Authority (RA). Subsequently, we initiated another key exchange process between Device A and Device B. As expected, the devices generated a new identical shared secret key, different from the previous key due to the updated key pair of Device A. This successful generation of a new shared secret key after the key update process, as shown in Table 5, further confirms the protocol's robustness and ability to maintain secure communications even after key updates.

These tests demonstrate that the proposed protocol effectively manages the registration, key exchange, and key update processes, ensuring secure communication between devices in an IoT environment. The protocol's ability to consistently produce identical shared secret keys during key exchanges and manage key updates without compromising security validates its practical applicability and reliability.

### 7.2.2. Performance Testing

In the performance testing phase, we evaluated the efficiency of the proposed protocol in terms of time complexity, memory complexity of the proposed protocol. The tests were conducted in a controlled environment with multiple IoT devices to assess the protocol's performance under real-world conditions.

**(i) Time Complexity:** The time complexity of the proposed protocol was evaluated based on the execution time required for three critical operations: device registration, key exchange, and key update. Each operation was performed 10 times, and the average execution time was recorded to ensure the reliability of the results. The time measurements include computational overhead, waiting time, and blockchain transaction delays, reflecting the real-world deployment conditions of the protocol.

The device registration process, which involves generating X25519 and Ed25519 key pairs, transmitting the registration request, and waiting time, took an average of

**Table 5**

Shared secret key generation after update.

| Device   | Key   | Value                                                            |
|----------|-------|------------------------------------------------------------------|
| A        | $Q_a$ | 10d19d0f6073ac0b059170fd10145a581e59873de1fa457df0af47c84f986969 |
|          | $P_a$ | 452160aa427eb33cdb560eea1c9d9128e47b35994af5302035ecb98b4619c52b |
| B        | $Q_b$ | a8bad0b7b0d07114eb2bccbd023cce4e507e533a568acb09e244e90885499461 |
|          | $P_b$ | a73b96bde95feb81bd1de6bd08986645e5f1d2860970b4c1af320e9d015c020  |
| A-B pair | $K_a$ | 4abb2de5b7e839476c8e3a736634f972b4a2137df490b08032bb6b8e828b1f54 |
|          | $K_b$ | 4abb2de5b7e839476c8e3a736634f972b4a2137df490b08032bb6b8e828b1f54 |

**Table 6**

Time complexity of Registration

| SN. | X25519 key Gen( $\mu$ s) | Ed25519 key gen( $\mu$ s) | Waiting ( $\mu$ s) | Total ( $\mu$ s) |
|-----|--------------------------|---------------------------|--------------------|------------------|
| 1   | 32591                    | 60040                     | 10895918           | 10988549         |
| 2   | 32551                    | 59955                     | 6725955            | 6818461          |
| 3   | 32552                    | 59961                     | 7822006            | 7914519          |
| 4   | 32549                    | 59950                     | 18598302           | 18690801         |
| 5   | 32548                    | 59960                     | 6702955            | 6795463          |
| 6   | 32564                    | 59975                     | 6402943            | 6495482          |
| 7   | 32549                    | 59960                     | 8664021            | 8756530          |
| 8   | 32550                    | 59958                     | 29649408           | 29741916         |
| 9   | 32554                    | 59962                     | 7320629            | 7413145          |
| 10  | 32549                    | 59958                     | 6234959            | 6327466          |
| Av: | 32555                    | 59967                     | 10901709           | <b>10994233</b>  |

**Table 7**

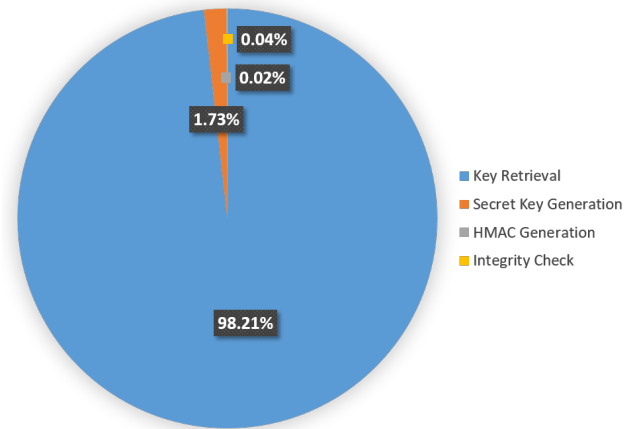
Time complexity of key Exchange

| SN. | Key Retrieval( $\mu$ s) | Secret Key Gen.( $\mu$ s) | HMAC Gen.( $\mu$ s) | Integrity Check( $\mu$ s) | Total ( $\mu$ s) |
|-----|-------------------------|---------------------------|---------------------|---------------------------|------------------|
| 1   | 1892356                 | 32479                     | 371                 | 753                       | 1925959          |
| 2   | 1973348                 | 32497                     | 369                 | 749                       | 2006963          |
| 3   | 1815191                 | 32475                     | 370                 | 740                       | 1848776          |
| 4   | 1922381                 | 32482                     | 370                 | 744                       | 1955977          |
| 5   | 1819364                 | 32470                     | 371                 | 749                       | 1852954          |
| 6   | 1750357                 | 32491                     | 371                 | 749                       | 1783968          |
| 7   | 1790344                 | 32478                     | 370                 | 741                       | 1823933          |
| 8   | 1832352                 | 32490                     | 371                 | 749                       | 1865962          |
| 9   | 1818322                 | 32472                     | 369                 | 748                       | 1851911          |
| 10  | 1833284                 | 32476                     | 370                 | 750                       | 1866880          |
| Av  | 1844730                 | 32481                     | 370                 | 747                       | <b>1878328</b>   |

10,994,233 microseconds (10.99 sec), as shown in Table 6. Among these steps, waiting time primarily consists of network transmission delays, the generation of a unique device identifier by the Registration Authority, and blockchain transaction processing. Since blockchain operations rely on a consensus mechanism and are asynchronous, their execution time varies based on network conditions and transaction validation speed. The cryptographic key pair generation adds computational overhead, but the primary source of delay stems from blockchain-related processes, ensuring secure and tamper-proof device registration.

For key exchange, the average time required by each device was 1,878,328 microseconds (approximately 1.8 seconds), as shown in Table 7. Since the initiator device must wait for the response from the responder, the total average time to establish a secure session becomes 3,756,656 microseconds (3.7 seconds). This operation consists of retrieving the public key from the blockchain, generating the shared secret using X25519, computing the HMAC for message integrity, and verifying the integrity of the received data. Additionally, waiting time occurs while the responder device performs the same computations before responding to the initiator. Compared to the registration process, key exchange is significantly faster since it does not require blockchain transactions, reducing delays caused by consensus mechanisms.

According to Table 7, public key retrieval from the blockchain takes an average of 98.21% of the total execution time, while the remaining cryptographic operations contribute negligibly to the overall duration, as illustrated in Figure 9.

**Figure 9:** Breakdown of Time Utilization During Key Exchange

The key update operation, which includes generating a new X25519 key pair, signing the update request, retrieving the Registration Authority's signature verification key from blockchain, and signature verification, took an average of 16,585,131 microseconds (16.58 sec), as shown

**Table 8**

Time complexity of key Update

| SN. | X25519 key gen ( $\mu$ s) | Signing ( $\mu$ s) | Waiting ( $\mu$ s) | key retrieval ( $\mu$ s) | Signature verification ( $\mu$ s) | Total ( $\mu$ s) |
|-----|---------------------------|--------------------|--------------------|--------------------------|-----------------------------------|------------------|
| 1   | 32583                     | 60635              | 7968904            | 1945204                  | 98700                             | 10106026         |
| 2   | 32533                     | 60450              | 17848874           | 2344513                  | 98696                             | 20385066         |
| 3   | 32541                     | 60454              | 17840354           | 2152014                  | 98696                             | 20184059         |
| 4   | 32536                     | 60449              | 18477881           | 2176429                  | 98697                             | 20845992         |
| 5   | 32532                     | 60451              | 5812911            | 1946454                  | 98694                             | 7951042          |
| 6   | 32536                     | 60447              | 5983681            | 2349621                  | 98696                             | 8524981          |
| 7   | 32532                     | 60449              | 16299885           | 1825458                  | 98697                             | 18317021         |
| 8   | 32550                     | 60442              | 19595876           | 2039440                  | 98699                             | 21827007         |
| 9   | 32535                     | 60448              | 17541864           | 2035490                  | 98696                             | 19769033         |
| 10  | 32530                     | 60444              | 15702894           | 2046522                  | 98699                             | 17941089         |
| Av: | 32540                     | 60466              | 14307312           | 2086114                  | 98697                             | <b>16585131</b>  |

**Table 9**

Blockchain network writing time for registration and update.

| Txn # | Registration ( $\mu$ s) | Update ( $\mu$ s) |
|-------|-------------------------|-------------------|
| 1     | 7940234                 | 3994812           |
| 2     | 3835572                 | 13663313          |
| 3     | 4843873                 | 13725841          |
| 4     | 15765657                | 14671586          |
| 5     | 3725514                 | 2061968           |
| 6     | 3539705                 | 2229909           |
| 7     | 5692230                 | 12550653          |
| 8     | 26784136                | 15839430          |
| 9     | 4315438                 | 13878445          |
| 10    | 3251091                 | 11906405          |
| Av    | 7969345                 | 10452236          |

in Table 8. Similar to registration, the blockchain transaction introduces additional waiting time, as it requires verifying the authenticity of the update request by the Registration Authority before committing the new key to the ledger. This verification process ensures that only authorized devices can update their keys, preventing unauthorized modifications. However, since blockchain transactions depend on consensus validation and network conditions, this step introduces a higher execution time compared to key exchange.

**(ii) Memory Complexity:** To evaluate the feasibility of deploying the proposed protocol on resource-constrained IoT devices, we analyzed its memory usage by compiling the firmware on an ESP32 using the Arduino IDE. The compilation results indicate that the firmware consumes 931,425 bytes (29%) of the available 3,145,728 bytes of program storage space (Flash memory), leaving sufficient room for additional functionality and future updates.

For dynamic memory (RAM) usage, the firmware utilizes 46,448 bytes (14%) of the available 327,680 bytes, leaving 281,232 bytes available for local variable storage and runtime operations. This ensures that the device has enough memory for handling cryptographic operations, blockchain interactions, and real-time communication without risking

**Table 10**

Size of stored information in IoT devices

| SN.   | Type                        | Size (in byte) |
|-------|-----------------------------|----------------|
| 1.    | ID                          | 8              |
| 2.    | X25519 Private Key ( $Q$ )  | 32             |
| 3.    | Ed25519 Private Key ( $N$ ) | 32             |
| Total |                             | 70             |

memory overflow or instability. The size of the stored information is shown in Table 10.

### 7.2.3. Security Testing

To evaluate the robustness of the proposed protocol, we conducted security testing based on the threat model described in Section 3. The testing focused on assessing the protocol's resilience against eavesdropping, impersonation, modification, and replay attacks. To perform real-world attack scenarios, we utilized a laptop running Kali Linux along with a Wi-Fi adapter supporting monitoring and packet injection mode. Each attack scenario was performed in a controlled environment, and the effectiveness of the implemented security mechanisms was analyzed.

The first step in the security evaluation involved capturing the communication between two IoT devices to determine whether an attacker could extract sensitive information necessary for a man-in-the-middle (MitM) attack. Using Wireshark, we analyzed the captured packets and confirmed that no sensitive information, such as public keys, private keys or session keys, was exposed during the key exchange process. As shown in Figure 10.

To evaluate the protocol's resistance to impersonation attacks, we attempted to use device information such as the device ID extracted from the captured packets. However, without access to the private key associated with the targeted device, the impersonation attempt was unsuccessful. The protocol's cryptographic authentication mechanism ensures that only legitimate devices possessing the correct private key can successfully establish communication, thereby mitigating the risk of identity spoofing.

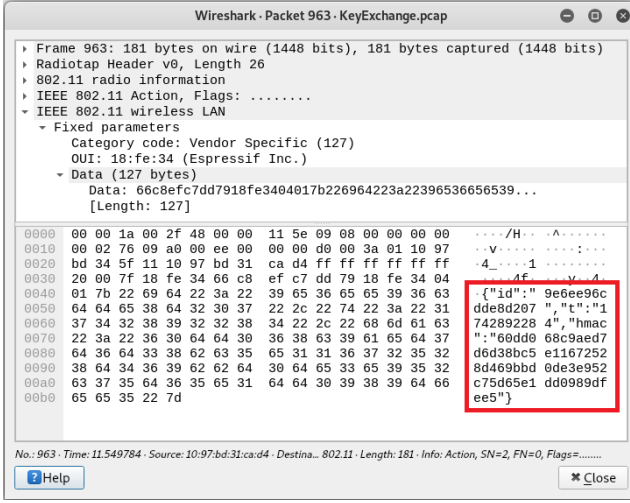


Figure 10: Captured packet analysis report

Next, we tested the protocol's resilience against replay attacks by resending previously captured packets to the receiver. The receiver successfully detected and rejected these replayed packets using timestamp validation, as depicted in Figure 11. This demonstrates that the protocol incorporates effective anti-replay measures to prevent adversaries from injecting previously recorded messages to manipulate communication.

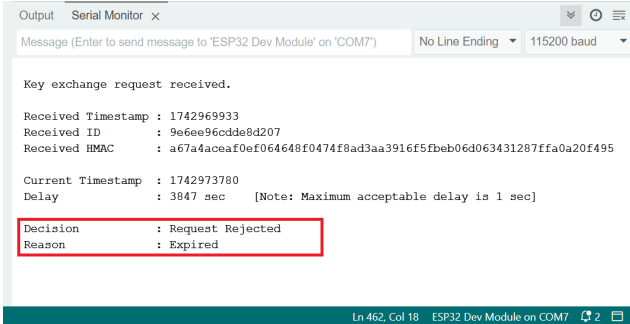


Figure 11: Replay attack detection

To further analyze the replay protection, we modified the packet's timestamp to reflect the current time and retransmitted it. Despite this alteration, the receiver correctly identified the modification attempt by verifying the HMAC (Hash-based Message Authentication Code), as shown in Figure 12. Since the integrity of each message is verified through HMAC, unauthorized modifications to packet contents are immediately detected, ensuring that attackers cannot bypass authentication mechanisms by simply updating timestamps or any other information.

To test the integrity protection of cryptographic key updates, we attempted to modify the public key of a device during an update request using Bettercap, a widely used penetration testing tool. The Registration Authority (RA), responsible for validating such requests, promptly detected the unauthorized modification and rejected the request, as

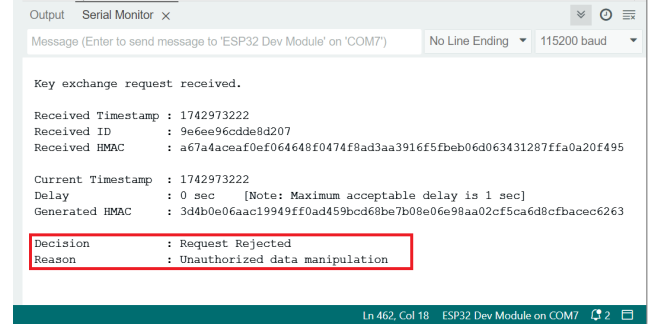


Figure 12: Modification attack detection

Table 11

Size of stored information in Blockchain

| SN.   | Type                       | Size (in byte) |
|-------|----------------------------|----------------|
| 1.    | ID                         | 8              |
| 2.    | X25519 Public Key ( $P$ )  | 32             |
| 3.    | Ed25519 Public Key ( $M$ ) | 32             |
| Total |                            | 70             |

illustrated in Figure 13. This confirms that the protocol enforces strict integrity checks, preventing adversaries from injecting malicious public keys into the system.

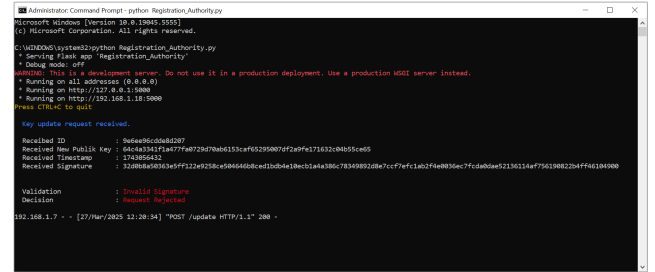


Figure 13: Modification attack detection on key update request

The security testing results confirm that the proposed protocol is highly resilient to common attack vectors in IoT communications. The use of cryptographic authentication, timestamp-based verification, and HMAC integrity checks ensures protection against eavesdropping, replay, impersonation, and modification attacks.

#### 7.2.4. Cost Analysis

To evaluate the cost-effectiveness of deploying the proposed protocol on blockchain infrastructure, we analyzed the transaction fees associated with two core operations: device registration and key update. This evaluation was conducted through practical testing on the Sepolia testnet and cost estimation for mainnet deployments on both Layer 1 (Ethereum) and Layer 2 (Polygon) blockchain networks.

According to the proposed protocol, device registration involves transmitting the device ID, X25519 public key, and Ed25519 public key, resulting in a total transaction data size of exactly 70 KB, as shown in Table 11. In contrast, the key

**Table 12**

Writing Gas Cost for Registration - Sepolia Testnet

| Txn       | Gas           | Gas Price(Gwei)    | Gas Price(ETH)              | Cost(ETH)                  |
|-----------|---------------|--------------------|-----------------------------|----------------------------|
| 1         | 186046        | 0.066933500        | 0.000000000066933500        | 0.000012452709941000       |
| 2         | 186046        | 0.074863976        | 0.000000000074863976        | 0.000013928143278896       |
| 3         | 186046        | 0.074595026        | 0.000000000074595026        | 0.000013878106207196       |
| 4         | 186046        | 0.076582025        | 0.000000000076582025        | 0.000014247779423150       |
| 5         | 186046        | 0.091007276        | 0.000000000091007276        | 0.000016931539670696       |
| 6         | 186046        | 0.087914946        | 0.000000000087914946        | 0.000016356224043516       |
| 7         | 186046        | 0.091130026        | 0.000000000091130026        | 0.000016954376817196       |
| 8         | 186046        | 0.110430368        | 0.000000000110430368        | 0.000020545128244928       |
| 9         | 186046        | 0.124026957        | 0.000000000124026957        | 0.000023074719242022       |
| 10        | 186046        | 0.111655085        | 0.000000000111655085        | 0.000020772981943910       |
| <b>Av</b> | <b>186046</b> | <b>0.090316019</b> | <b>0.000000000090316019</b> | <b>0.00001691457098183</b> |

**Table 13**

Writing Gas Cost for Key Update - Sepolia Testnet

| Txn       | Gas          | Gas Price(Gwei)    | Gas Price(ETH)              | Cost(ETH)                   |
|-----------|--------------|--------------------|-----------------------------|-----------------------------|
| 1         | 42973        | 0.124822700        | 0.000000000124822700        | 0.000005364005887100        |
| 2         | 42973        | 0.122551754        | 0.000000000122551754        | 0.000005266416524642        |
| 3         | 42973        | 0.117781820        | 0.000000000117781820        | 0.000005061438150860        |
| 4         | 42973        | 0.154714004        | 0.000000000154714004        | 0.000006648524893892        |
| 5         | 42973        | 0.190807154        | 0.000000000190807154        | 0.000008199555828842        |
| 6         | 42973        | 0.195074706        | 0.000000000195074706        | 0.000008382945340938        |
| 7         | 42973        | 0.254951868        | 0.000000000254951868        | 0.000010956046623564        |
| 8         | 42973        | 0.344232883        | 0.000000000344232883        | 0.000014792719681159        |
| 9         | 42973        | 0.306017592        | 0.000000000306017592        | 0.000013150493981016        |
| 10        | 42973        | 0.310666051        | 0.000000000310666051        | 0.000013350252209624        |
| <b>Av</b> | <b>42973</b> | <b>0.212461453</b> | <b>0.000000000212461453</b> | <b>0.000009417239012564</b> |

update operation requires only the device ID and the new X25519 public key, with a fixed data size of 40 KB.

The transaction cost is primarily influenced by the gas amount, which depends on the size of the data and the computational resources required to execute smart contract functions. Based on transactions conducted on the Sepolia test network, the gas amount for device registration is 186,046, while key update operations require only 42,973 units of gas. This disparity indicates that registration consumes more gas due to the storage of new data on the blockchain, whereas key updates are more efficient, as they involve modifying existing records rather than inserting new entries.

To empirically evaluate the transaction costs of the proposed protocol, we conducted 10 device registration and 10 key update transactions on the Sepolia testnet. The resulting costs are summarized in Table 12 and Table 13, respectively.

On average, device registration incurred a cost of approximately 0.0000169 ETH, while key updates required a lower cost of about .0000094 ETH. These testnet-based results offer a preliminary understanding of the gas requirements and associated costs for implementing the protocol in a live blockchain environment.

In addition to the testnet experiments, we evaluated the transaction cost of maintaining the proposed protocol on blockchain mainnets. Specifically, we analyzed Ethereum

as a Layer1 network and Polygon as a Layer2 network. To perform this analysis, we collected the average daily gas prices over the past 100 days (from March 10, 2025, to June 17, 2025) in wei, obtained from Etherscan[40] for Ethereum and Polygonscan[41] for Polygon.

The collected gas prices were then converted from wei to their respective native tokens (ETH for Ethereum and MATIC for Polygon), as illustrated in Figure 14. To determine the monetary value, we retrieved the corresponding daily token prices in USD from CoinGecko[42], as shown in Figure 15. Using these values, we computed the effective gas price in USD for both networks, as presented in Figure 16. Finally, by multiplying the USD-denominated gas price by the gas amount required for each operation (device registration and key update), we estimated the transaction costs for each network. The average cost for device registration is presented in Figure 17, while the cost for key updates is shown in Figure 18.

Based on the actual amount of gas consumed, the average transaction cost for device registration is approximately \$1.3313214748 on Ethereum and \$0.0037676082 on Polygon, as illustrated in Figure 17. In contrast, the average transaction cost for key updates is approximately \$0.3075093135 on Ethereum and \$0.0008702441 on Polygon, as shown in Figure 18. These results highlight the cost efficiency of the

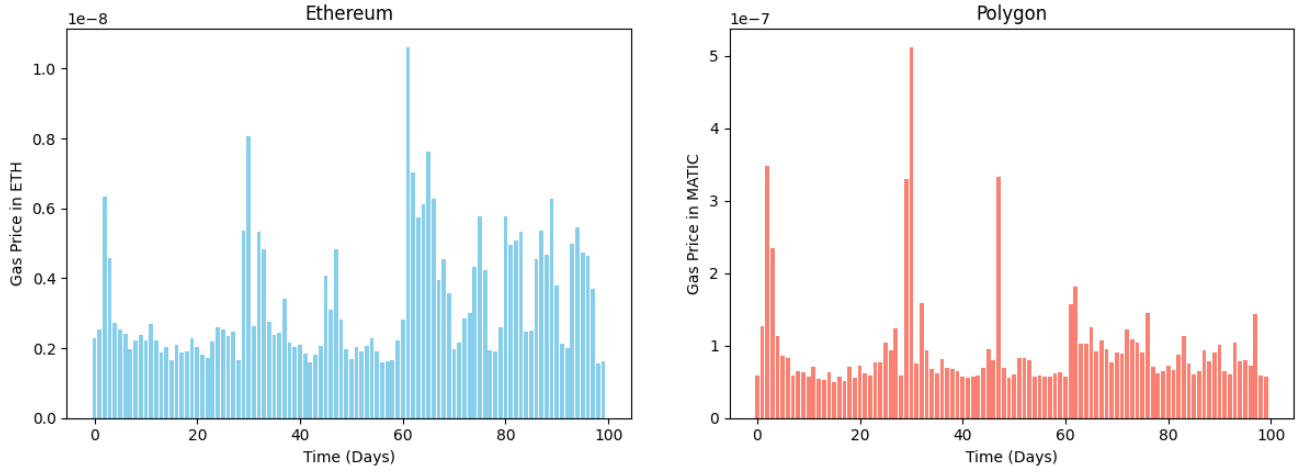


Figure 14: Gas price in token

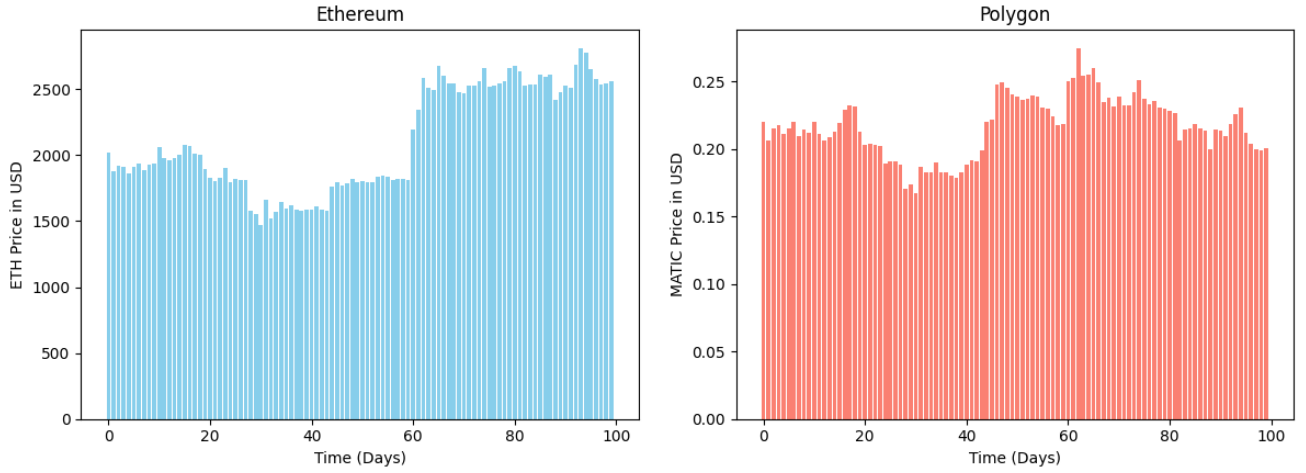


Figure 15: Token price in USD

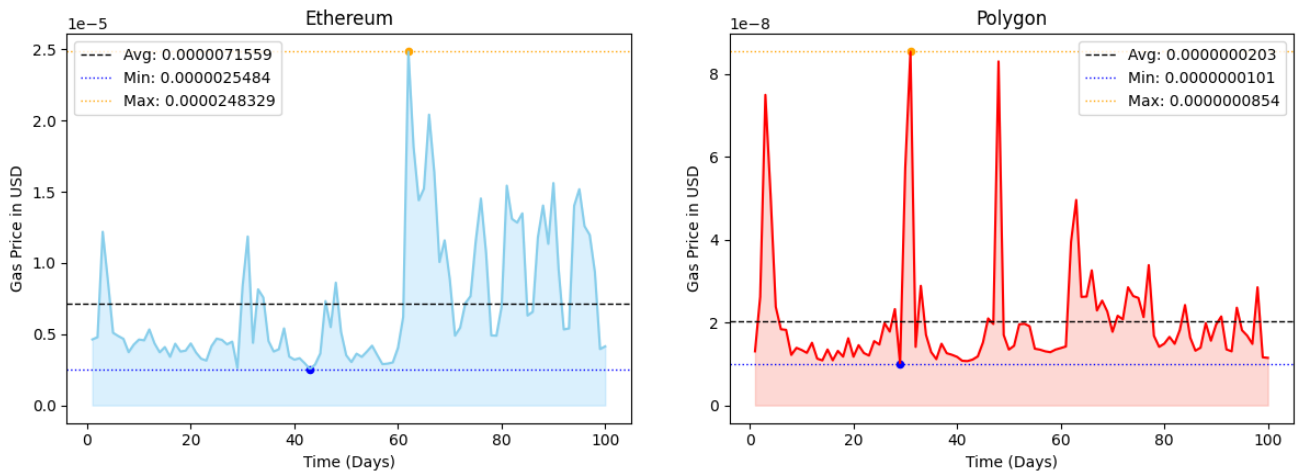


Figure 16: Gas price in USD

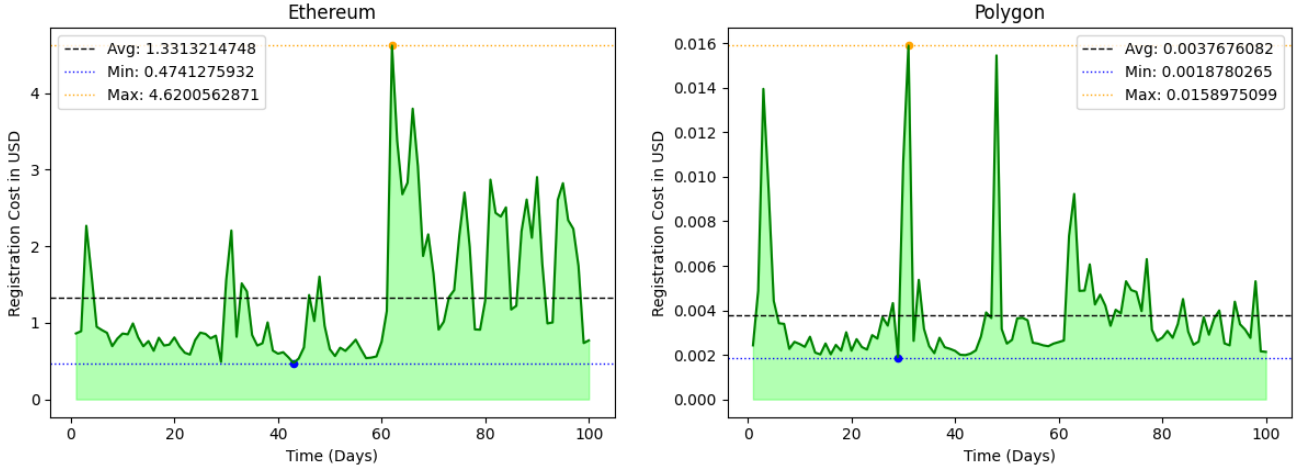


Figure 17: Registration cost in USD (186046 Unit Gas)

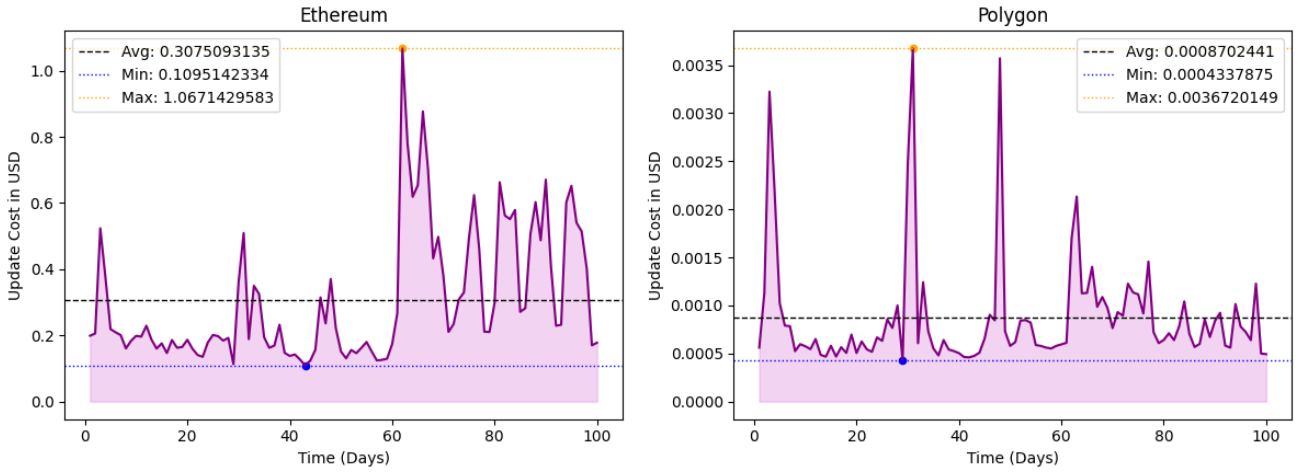


Figure 18: Update cost in USD (42973 Unit Gas)

protocol, particularly when deployed on Layer 2 solutions like Polygon.

These findings highlight the cost-effectiveness and practical feasibility of deploying the proposed blockchain-based key management protocol in IoT environments.

## 8. Discussion

The proposed blockchain and elliptic curve cryptography (ECC)-based lightweight key exchange protocol has been specifically designed to address the security and efficiency challenges of resource-constrained IoT devices. To evaluate its feasibility and lightweight nature, we implemented the protocol on ESP32 microcontrollers and analyzed various performance metrics, including time complexity, memory usage, power consumption, and blockchain transaction costs. The results demonstrate that the protocol is both computationally efficient and feasible for deployment on low-power IoT devices. Additionally, we conducted a

formal security analysis using the Scyther tool, performed real-world security testing against common attacks, and carried out an informal evaluation of the protocol's security strength.

The latency of the proposed protocol is significantly influenced by the consensus mechanism and performance characteristics of the underlying blockchain network. In our experiments conducted on the Sepolia testnet, results indicate that public key retrieval from the blockchain requires approximately 98.21% of the total time needed for the key exchange process. This indicates that the speed and efficiency of the blockchain network play a critical role in determining the overall latency of secure session establishment. Consequently, utilizing faster blockchain networks or Layer 2 solutions can substantially reduce communication delays, while slower networks may introduce noticeable latency. Optimizing blockchain interaction is therefore essential for improving the protocol's responsiveness, especially in time-sensitive IoT applications.

**Table 14**  
Protocol comparison

| Year | Paper                    | Computational Cost | Memory Requirement | Hardware Utilization | Central Intermediaries | Compatible in Constrained IoT Device |
|------|--------------------------|--------------------|--------------------|----------------------|------------------------|--------------------------------------|
| 2021 | Zheng et al. [5]         | Low                | Low                | PUF                  | Required               | Yes                                  |
| 2022 | Roy et al. [6]           | Low                | Low                | PUF                  | Required               | Yes                                  |
| 2023 | Ashraf et al. [16]       | Low                | Low                | -                    | Required               | Yes                                  |
| 2024 | Abdussami et al. [7]     | Low                | Low                | FPGA                 | Not Required           | Yes                                  |
| 2024 | Fan et al. [8]           | Low                | Low                | PUF                  | Not Required           | Yes                                  |
| 2024 | Sarkar et al. [9]        | High               | High               | -                    | Not Required           | No                                   |
| 2024 | Selvakumar et al. [10]   | High               | High               | -                    | Not Required           | No                                   |
|      | <b>Proposed Protocol</b> | <b>Low</b>         | <b>Low</b>         | <b>-</b>             | <b>Not Required</b>    | <b>Yes</b>                           |

When compared with other recent key exchange protocols, as presented in Table 14, several existing approaches rely on specialized hardware components such as Physically Unclonable Functions (PUF) and Field-Programmable Gate Arrays (FPGA). While these hardware-based security mechanisms offer strong protection, their integration into IoT devices is challenging due to increased hardware complexity and additional manufacturing costs. Implementing these specialized chips in resource-constrained IoT devices is impractical for large-scale deployment, making software-based security solutions like the proposed protocol more viable.

Some existing protocols also focus on quantum resistance, ensuring security against future quantum computing threats. However, these protocols often require higher computational resources, making them unsuitable for lightweight devices like ESP32. Quantum-resistant cryptographic algorithms tend to be complex and demand significant processing power, memory, and energy, making them incompatible with low-power IoT environments. In contrast, the proposed protocol leverages ECC, which provides strong security with smaller key sizes and lower computational overhead, making it well-suited for resource-limited devices.

Another limitation of many existing protocols is their reliance on centralized intermediaries for authentication. Such dependency introduces a single point of failure, meaning that if the central authority becomes unavailable, the entire system can be disrupted. The proposed protocol eliminates this issue by leveraging blockchain technology to store and verify device public keys. Since blockchain operates in a decentralized manner, there is no need for a central authentication server, thereby enhancing system reliability and resilience against central authority failures.

Despite its advantages, the proposed protocol has certain limitations. One major drawback is that it is not quantum-resistant. As quantum computing technology advances, traditional ECC-based cryptographic schemes may become vulnerable to attacks by quantum algorithms such as Shor's algorithm. Future research should explore post-quantum cryptographic techniques that maintain compatibility with resource-constrained IoT devices while ensuring resistance to quantum attacks.

Another limitation is the role of the registration authority in the system. The registration authority holds significant control over the network, including the ability to add new devices and modify blockchain-stored information. If the registration authority is compromised, an attacker could potentially manipulate device registrations or introduce malicious modifications, putting the entire system at risk. Therefore, ensuring the security of the registration authority is crucial. Both physical security measures and robust cybersecurity protections must be implemented to prevent unauthorized access and potential attacks. Future improvements could consider decentralized identity management mechanisms to reduce dependency on a single trusted entity.

In conclusion, the proposed protocol provides a secure, lightweight, and decentralized key exchange mechanism suitable for IoT environments. It effectively mitigates common security threats while maintaining compatibility with constrained devices. However, addressing quantum resistance and enhancing the security of the registration authority are important areas for future enhancement to further strengthen the protocol's robustness.

## 9. Conclusion

Device-to-device (D2D) communication in the Internet of Things (IoT) enables direct and efficient interaction between connected devices, supporting a diverse range of applications across industries such as autonomous and connected vehicles, smart healthcare, industrial automation, financial systems, and intelligent transportation. These systems frequently process and transmit sensitive information including personal data, financial records, and telemetry from vehicles necessitating robust mechanisms for secure communication. However, due to the inherent limitations of IoT devices in terms of processing power, storage capacity, and energy efficiency, conventional approaches typically rely on centralized servers or coordination mechanisms for key management and authentication. This reliance introduces risks such as single points of failure, and operational overhead, additional costs, and others. As a remedy, blockchain technology presents a decentralized and tamper-resistant alternative that removes the need for centralized trust and offers enhanced fault tolerance.

The proposed protocol combines elliptic curve cryptography with blockchain-based public key management to facilitate secure session key generation and mutual authentication without the involvement of centralized intermediaries. By leveraging a distributed ledger, devices can retrieve verified public keys securely, eliminating the need to exchange keys directly over potentially insecure channels.

We validated our protocol through practical implementation on ESP32 devices to evaluate its feasibility and performance under resource-constrained conditions. Experimental results demonstrate the protocol's efficiency in terms of time complexity and memory footprint. Security testing, covering attacks such as man-in-the-middle (MITM), impersonation, replay, and message modification, confirms the robustness of the proposed approach. Moreover, formal verification using the Scyther tool, alongside informal security analysis, shows strong guarantees for integrity, authenticity, and forward secrecy. Cost analysis further affirms the protocol's viability for deployment over both Layer 1 (Ethereum) and Layer 2 (Polygon) blockchain infrastructures.

This research contributes a significant advancement toward secure D2D communication by removing the dependency on specialized hardware components and centralized authentication systems. It provides a scalable, decentralized, and lightweight security solution suitable for modern IoT ecosystems. However, the protocol still relies on a trusted registration authority and does not currently address quantum-resistance, representing areas for future enhancement.

Future work will focus on extending this protocol into a comprehensive secure communication framework for D2D interactions and evaluating its performance in large-scale and real-world IoT environments. Such developments will further strengthen the scalability, trust, and applicability of secure communication in decentralized IoT systems.

## CRedit authorship contribution statement

**Rakibul Hasan:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft. **Md Al Amin:** Supervision, Resources, Validation, Writing – review and editing. **Md Ibrahim Abdullah:** Validation, Writing – review and editing. **A B M Shawkat Ali:** Resources, Writing – review and editing.

## References

- [1] S. Kumar, P. Tiwari, M. Zymbler, Internet of things is a revolutionary approach for future technology enhancement: a review, *Journal of Big data* 6 (2019) 1–21.
- [2] L. Wang, L. Liu, X. Cao, X. Tian, Y. Cheng, Sociality-aware resource allocation for device-to-device communications in cellular networks, *IET Communications* 9 (2015) 342–349.
- [3] P. Pawar, A. Trivedi, Device-to-device communication based iot system: benefits and challenges, *IETE Technical Review* 36 (2019) 362–374.
- [4] I. S. Udoh, G. Kotonya, Developing iot applications: challenges and frameworks, *IET Cyber-Physical Systems: Theory & Applications* 3 (2018) 65–72.
- [5] Y. Zheng, C.-H. Chang, Secure mutual authentication and key-exchange protocol between puf-embedded iot endpoints, in: 2021 IEEE International Symposium on Circuits and Systems (ISCAS), IEEE, 2021, pp. 1–5. doi:<https://doi.org/10.1109/ISCAS51556.2021.9401135>.
- [6] S. Roy, D. Das, A. Mondal, M. H. Mahalat, B. Sen, B. Sikdar, Plake: Puf-based secure lightweight authentication and key exchange protocol for iot, *IEEE Internet of Things Journal* 10 (2022) 8547–8559.
- [7] M. Abdussami, S. K. Dwivedi, T. Al-Shehari, P. Saravanan, M. Kadrie, T. Alfakih, H. Alsaman, R. Amin, Deac-iot: Design of lightweight authenticated key agreement protocol for intra and inter-iot device communication using ecc with fpga implementation, *Computers and Electrical Engineering* 120 (2024) 109696.
- [8] C.-I. Fan, C.-I. Lai, D. V. Medhane, Cake-puf: A collaborative authentication and key exchange protocol based on physically unclonable functions for industrial internet of things, *IEEE Internet of Things Journal* (2024).
- [9] P. Sarkar, A. Nag, Lattice-based device-to-device authentication and key exchange protocol for iot system, *International Journal of Information Technology* 16 (2024) 4167–4179.
- [10] S. Selvakumar, A. Ahilan, B. Ben Sujitha, N. Muthukumaran, Crystals kyber cryptographic algorithm for efficient iot d2d communication, *Wireless Networks* (2024) 1–18.
- [11] R. Hasan, M. Al Amin, S. D. Hossain, M. I. Abdullah, Blockchain-based key sharing mechanism for iot device-to-device (d2d) secure communications, in: 2023 IEEE International Conference on Telecommunications and Photonics (ICTP), IEEE, 2023, pp. 01–05. doi:<https://doi.org/10.1109/ICTP60248.2023.10490945>.
- [12] M. N. Aman, K. C. Chua, B. Sikdar, A light-weight mutual authentication protocol for iot systems, in: GLOBECOM 2017-2017 IEEE Global Communications Conference, IEEE, 2017, pp. 1–6. doi:<https://doi.org/10.1109/GLOCOM.2017.8253991>.
- [13] U. Chatterjee, V. Govindan, R. Sadhukhan, D. Mukhopadhyay, R. S. Chakraborty, D. Mahata, M. M. Prabhu, Building puf based authentication and key exchange protocol for iot without explicit crps in verifier database, *IEEE transactions on dependable and secure computing* 16 (2018) 424–437.
- [14] S. Li, T. Zhang, B. Yu, K. He, A provably secure and practical puf-based end-to-end mutual authentication and key exchange protocol for iot, *IEEE Sensors Journal* 21 (2020) 5487–5501.
- [15] C. Gupta, G. Varshney, A lightweight and secure puf-based authentication and key-exchange protocol for iot devices, *arXiv preprint arXiv:2311.04078* (2023).
- [16] Z. Ashraf, A. Sohail, M. Yousaf, Robust and lightweight symmetric key exchange algorithm for next-generation ioe, *Internet of Things* 22 (2023) 100703.
- [17] A. B. Rabiah, K. Ramakrishnan, E. Liri, K. Kar, A lightweight authentication and key exchange protocol for iot, in: Workshop on Decentralized IoT Security and Standards, volume 2018, sn, 2018, pp. 1–6. doi:<https://dx.doi.org/10.14722/diss.2018.23004>.
- [18] B. B. Ehui, Y. Han, H. Guo, J. Liu, A lightweight mutual authentication protocol for iot, *Journal of Communications and Information Networks* 7 (2022) 181–191.
- [19] Q. Fan, J. Chen, M. Shojafar, S. Kumari, D. He, Sake\*: A symmetric authenticated key exchange protocol with perfect forward secrecy for industrial internet of things, *IEEE transactions on industrial informatics* 18 (2022) 6424–6434.
- [20] W. Stallings, *Cryptography and Network Security Principles and Practice*, sixth ed., New Jersey: Pearson Education, Inc., 2014.
- [21] R. K. Kodali, A. Naikoti, Ecdh based security model for iot using esp8266, in: 2016 International Conference on Control, Instrumentation, Communication and Computational Technologies (ICCICCT), IEEE, 2016, pp. 629–633. doi:<https://doi.org/10.1109/ICCICCT.2016.7988026>.
- [22] I. Ali, S. Sabir, Z. Ullah, Internet of things security, device authentication and access control: a review, *International Journal of Computer Science and Information Security* 14 (2016).

- [23] A. Phimphinith, X. Anping, Q. Zhu, Y. Jiang, Y. Shen, An enhanced mutual authentication scheme based on ecdh for iot devices using esp8266, in: 2019 IEEE 11th International Conference on Communication Software and Networks (ICCSN), IEEE, 2019, pp. 490–496. doi:<https://doi.org/10.1109/ICCSN.2019.8905268>.
- [24] R. O. Sinnott, D. W. Chadwick, T. Doherty, D. Martin, A. Stell, G. Stewart, L. Su, J. Watt, Advanced security for virtual organizations: The pros and cons of centralized vs decentralized security models, in: 2008 Eighth IEEE International Symposium on Cluster Computing and the Grid (CCGRID), IEEE, 2008, pp. 106–113. doi:<https://doi.org/10.1109/CCGRID.2008.67>.
- [25] Y. Bandung, et al., Design of secure iot platform for smart home system, in: 2018 5th International Conference on Information Technology, Computer, and Electrical Engineering (ICITACEE), IEEE, 2018, pp. 114–119. doi:<https://doi.org/10.1109/ICITACEE.2018.8576932>.
- [26] O. Salman, S. Abdallah, I. H. Elhaji, A. Chehab, A. Kayssi, Identity-based authentication scheme for the internet of things, in: 2016 IEEE Symposium on Computers and Communication (ISCC), IEEE, 2016, pp. 1109–1111. doi:<https://doi.org/10.1109/ISCC.2016.7543884>.
- [27] T. Shah, S. Venkatesan, Authentication of iot device and iot server using secure vaults, in: 2018 17th IEEE international conference on trust, security and privacy in computing and communications/12th IEEE international conference on big data science and engineering (TrustCom/BigDataSE), IEEE, 2018, pp. 819–824. doi:<https://doi.org/10.1109/TrustCom/BigDataSE.2018.00117>.
- [28] G. S. Gaba, G. Kumar, H. Monga, T.-H. Kim, M. Liyanage, P. Kumar, Robust and lightweight key exchange (lke) protocol for industry 4.0, IEEE Access 8 (2020) 132808–132824.
- [29] R. Muth, F. Tschorsch, Smartdhtx: Diffie-hellman key exchange with smart contracts, in: 2020 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS), IEEE, 2020, pp. 164–168. doi:<https://doi.org/10.1109/DAPPS49028.2020.00022>.
- [30] M. El-Hajj, A. Fadlallah, M. Chamoun, A. Serhrouchni, Ethereum for secure authentication of iot using pre-shared keys (psks), in: 2019 International Conference on Wireless Networks and Mobile Communications (WINCOM), IEEE, 2019, pp. 1–7. doi:<https://doi.org/10.1109/WINCOM47513.2019.8942487>.
- [31] P. Koppermann, F. De Santis, J. Heyszl, G. Sigl, X25519 hardware implementation for low-latency applications, in: 2016 Euromicro Conference on Digital System Design (DSD), IEEE, 2016, pp. 99–106. doi:<https://doi.org/10.1109/DSD.2016.65>.
- [32] D. J. Bernstein, X25519 naming, posting to the cfrg mailing list., [https://mailarchive.ietf.org/arch/msg/cfrg/-9LEdnzVrE5R0Rux30o\\_oDDRksU/](https://mailarchive.ietf.org/arch/msg/cfrg/-9LEdnzVrE5R0Rux30o_oDDRksU/), 2014. [Accessed 22-03-2025].
- [33] Y. Zhang, Y. Shen, H. Wang, J. Yong, X. Jiang, On secure wireless communications for iot under eavesdropper collusion, IEEE Transactions on Automation Science and Engineering 13 (2015) 1281–1293.
- [34] O. Salem, K. Alsubhi, A. Shaafi, M. Gheryani, A. Mehaoua, R. Boutaba, Man-in-the-middle attack mitigation in internet of medical things, IEEE Transactions on Industrial Informatics 18 (2021) 2053–2062.
- [35] Y. Shah, S. Sengupta, A survey on classification of cyber-attacks on iot and iiot devices, in: 2020 11th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), IEEE, 2020, pp. 0406–0413. doi:<https://doi.org/10.1109/UEMCON51285.2020.9298138>.
- [36] M. S. Wara, Q. Yu, New replay attacks on zigbee devices for internet-of-things (iot) applications, in: 2020 IEEE International Conference on Embedded Software and Systems (ICCESS), IEEE, 2020, pp. 1–6. doi:<https://doi.org/10.1109/ICCESS49830.2020.9301593>.
- [37] D. D. N. Nguyen, K. Sood, Y. Xiang, L. Gao, L. Chi, Impersonation attack detection in iot networks, in: GLOBECOM 2022-2022 IEEE Global Communications Conference, IEEE, 2022, pp. 6061–6066. doi:<https://doi.org/10.1109/GLOBECOM48099.2022.10001392>.
- [38] M. Düll, B. Haase, G. Hinterwälder, M. Hutter, C. Paar, A. H. Sánchez, P. Schwabe, High-speed curve25519 on 8-bit, 16-bit, and 32-bit microcontrollers, Designs, Codes and Cryptography 77 (2015) 493–514.
- [39] C. J. Cremers, The scyther tool: Verification, falsification, and analysis of security protocols: Tool paper, in: International conference on computer aided verification, Springer, 2008, pp. 414–418. doi:[https://doi.org/10.1007/978-3-540-70545-1\\_38](https://doi.org/10.1007/978-3-540-70545-1_38).
- [40] Etherscan, <https://etherscan.io/chart/gasprice>, ??? [Accessed 18-06-2025].
- [41] Polygonscan, <https://polygonscan.com/chart/gasprice>, ??? [Accessed 18-06-2025].
- [42] Cryptocurrency Prices, Charts, and Crypto Market Cap | CoinGecko, <https://www.coingecko.com/>, ??? [Accessed 18-06-2025].

**Declaration of interests**

☒The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☐The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: